⚡ ELECTRICAL TO SOFTWARE ARCHITECT EDITION

সিএস ডিগ্রি ছাড়া এআই দিয়ে সফটওয়্যার মাস্টার

নন-সিএস ও ইইই ইঞ্জিনিয়ারদের জন্য সফটওয়্যার আর্কিটেকচার, ওয়েব, মোবাইল অ্যাপ, ডকার, সিআই/সিডি, ক্লাউড ও এআই-চালিত ডেভেলপমেন্টের পূর্ণাঙ্গ হ্যান্ডবুক।

🖥️ কোর সিএস কনসেপ্টস
🐳 ডকার ও কনটেইনার
⚙️ গিটহাব অ্যাকশনস ও CI/CD
🌐 ওয়েব ও ক্লাউড আর্কিটেকচার
🤖 AI প্রম্পটিং আর্কিটেকচার

প্রকৌশলী ও সফটওয়্যার আর্কিটেক্টদের জন্য বিশেষ গাইড

বাস্তব উদাহরণ, হার্ডওয়্যার অ্যানালজি ও সহজ বাংলায় সম্পূর্ণ গাইড

"যন্ত্রের সার্কিট থেকে কোডের ক্লাউড — আপনার নতুন দিগন্ত"

INTRODUCTION

ভূমিকা: ইইই থেকে সফটওয়্যার আর্কিটেক্ট

সিএস ডিগ্রি ছাড়াই কীভাবে আধুনিক এআই টুলের সাহায্যে পূর্ণাঙ্গ সফটওয়্যার তৈরি করবেন

আপনার জন্য কেন এই বই?

আপনি ইলেকট্রিক্যাল অ্যান্ড ইলেকট্রনিক ইঞ্জিনিয়ারিং (EEE) শেষ করেছেন। সার্কিট থিওরি, ট্রানজিস্টর, ও'মস ল, সিগন্যাল প্রসেসিং, মাইক্রোকন্ট্রোলার কিংবা ইলেক্ট্রোম্যাগনেটিজম আপনার পরিচিত জগত। গণিত, লজিক আর সিস্টেম ডিজাইন আপনার রক্তে আছে। কিন্তু যখন আপনি আধুনিক কৃত্রিম বুদ্ধিমত্তা (AI) যেমন ChatGPT, Claude, Cursor বা GitHub Copilot ব্যবহার করে কোনো পূর্ণাঙ্গ সফটওয়্যার, মোবাইল অ্যাপ কিংবা ওয়েবসাইট বানাতে যান—তখনই সামনে এসে দাঁড়ায় এক বিশাল অদৃশ্য দেয়াল!

দেয়ালটির নাম "সিএস পরিভাষা ও সফটওয়্যার আর্কিটেকচার"।

এআই আপনাকে নিমিষেই ১০০০ লাইনের নিখুঁত কোড লিখে দিতে পারে। কিন্তু এআই যখন বলে: "আপনার অ্যাপটি ডকারাইজ (Dockerize) করে গিটহাব অ্যাকশনস (GitHub Actions) দিয়ে সিআই/সিডি পাইপলাইনে ডিপ্লয় করুন, রিভার্স প্রক্সি হিসেবে Nginx ব্যবহার করুন এবং ডাটাবেজ মাইগ্রেশনের জন্য Prisma চালান"—তখন মনে প্রশ্ন জাগে:

- ডকার (Docker) কি? এটা কি কোনো ভার্চুয়াল মেশিন নাকি অন্য কিছু?
- গিটহাব অ্যাকশনস (GitHub Actions) কি? গিটহাবে কোড রাখা ছাড়া আবার কী অ্যাকশন হয়?
- CORS এরর কেন আসছে? ফ্রন্টএন্ড আর ব্যাকএন্ড একই পিসিতে চলছে তাও কেন ডেটা পায় না?
- এপিআই (API), জেএসন (JSON), জেডব্লিউটি (JWT), ওআরএম (ORM), এসকিউএল (SQL) বনাম নো-এসকিউএল (NoSQL)—এই হাজারটা শব্দ আসলে কী বোঝায়?

> ⚡ একজন ইইই ইঞ্জিনিয়ারের সবচেয়ে বড় সুপারপাওয়ার
>
> কম্পিউটার সায়েন্স আসলে আকাশ থেকে পড়েনি। এটি ইলেকট্রিক্যাল সার্কিটের লজিক গেট (AND, OR, NOT) এবং সেমিকন্ডাক্টরের ওপর দাঁড়িয়ে থাকা বিমূর্ত (Abstraction) এক স্থাপত্য মাত্র। আপনি যদি কার্শফের ভোল্টেজ ল (KVL) বা অপ-অ্যাম্প সার্কিট বুঝতে পারেন, তবে সফটওয়্যার আর্কিটেকচার বোঝা আপনার জন্য সময়ের ব্যাপার মাত্র। সফটওয়্যারের প্রতিটি কনসেপ্টের পেছনেই হার্ডওয়্যার বা সিস্টেম ইঞ্জিনিয়ারিংয়ের হুবহু প্রতিচ্ছবি রয়েছে!

এআই যুগে আপনার ভূমিকা কী?

আজকের দিনে প্রতিটি কোডের সিনট্যাক্স মুখস্থ রাখা অর্থহীন। কারণ সিনট্যাক্স লেখার কাজ এআই সবচেয়ে দ্রুত ও নিখুঁতভাবে করতে পারে। আপনার আসল কাজ হলো সফটওয়্যার সিস্টেম আর্কিটেক্ট (Software System Architect) হওয়া। একজন সিভিল ইঞ্জিনিয়ার যেমন নিজে ইট-বালু মাথায় করে বহন করেন না, বরং ব্লুপ্রিন্ট ও স্ট্রাকচার ডিজাইন করেন—ঠিক তেমনি আপনার কাজ হলো সফটওয়্যার সিস্টেমের বিভিন্ন অংশ কীভাবে একে অপরের সাথে যুক্ত হবে তা বোঝা এবং এআই-কে সঠিক নির্দেশ দেওয়া।

> 🤖 এআই-এর সাথে কাজ করার মূল মন্ত্র
>
> "If you can name it and frame it, AI can build it." অর্থাৎ, আপনি যদি কোনো প্রযুক্তি বা আর্কিটেকচারের সঠিক নাম জানেন এবং তার কাজের পরিধি বুঝতে পারেন, তবে এআই দিয়ে সেটি বাস্তবায়ন করা কয়েক মিনিটের কাজ। এই বইটি আপনাকে সেই নাম, কাঠামো এবং গভীর বোঝাপড়া উপহার দেবে।

এই বইটি কীভাবে পড়বেন?

বইটি তাত্ত্বিক কোনো পাঠ্যবই নয়; এটি একটি বাস্তবসম্মত যুদ্ধের হ্যান্ডবুক। প্রতিটি অধ্যায়ে থাকবে:

1. মৌলিক ধারণা: জটিল পরিভাষার সহজ বাংলা ব্যাখ্যা।
2. হার্ডওয়্যার ও বাস্তব জীবনের অ্যানালজি: আপনার পরিচিত ইঞ্জিনিয়ারিং জগতের সাথে তুলনা।
3. বাস্তব উদাহরণ ও আর্কিটেকচার ফ্লো: সিস্টেমে এটি কীভাবে কাজ করে।
4. সাধারণ ভুল ও ডিব্যাগিং টিপস: এআই ব্যবহারের সময় কোথায় আটকাবেন এবং কীভাবে উদ্ধার পাবেন।
5. প্র্যাকটিক্যাল এআই প্রম্পট রেসিপি: ওই টপিক নিয়ে এআই-কে দিয়ে কাজ করিয়ে নেওয়ার প্রস্তুত প্রম্পট।

সূচিপত্র (Table of Contents)

অধ্যায় ১	সফটওয়্যার কীভাবে কাজ করে? কম্পিউটারের অন্দরমহল ও ফান্ডামেন্টালস	পৃষ্ঠা ৫
অধ্যায় ২	আধুনিক সফটওয়্যার আর্কিটেকচার: ওয়েব ও মোবাইল অ্যাপের কাঠামো	পৃষ্ঠা ৮
অধ্যায় ৩	নেটওয়ার্কিং, ইন্টারনেট ও এপিআই (APIs) এর নাড়িনক্ষত্র	পৃষ্ঠা ১১
অধ্যায় ৪	ডেটাবেজ ও স্টোরেজ ম্যানেজমেন্ট: তথ্য সংরক্ষণের কলাকৌশল	পৃষ্ঠা ১৫
অধ্যায় ৫	প্যাকেজ ইকোসিস্টেম, ডিপেন্ডেন্সি ও মডিউল সিস্টেম	পৃষ্ঠা ১৮
অধ্যায় ৬	ভার্সন কন্ট্রোল ও কোড কোলাবোরেশন: গিট (Git) ও গিটহাব (GitHub)	পৃষ্ঠা ২১
অধ্যায় ৭	কন্টেইনারাইজেশন ও ডকার (Docker): "আমার পিসিতে চলে কিন্তু সার্ভারে চলে না"র অবসান	পৃষ্ঠা ২৪
অধ্যায় ৮	অটোমেশন ও সিআই/সিডি: গিটহাব অ্যাকশনস (GitHub Actions)	পৃষ্ঠা ২৮
অধ্যায় ৯	ক্লাউড কম্পিউটিং, সার্ভার কনফিগারেশন ও লাইভ ডেপ্লয়মেন্ট	পৃষ্ঠা ৩১
অধ্যায় ১০	সিস্টেম ডিজাইন ও সফটওয়্যার আর্কিটেকচার প্যাটার্নস	পৃষ্ঠা ৩৪
অধ্যায় ১১	এআই দিয়ে সফটওয়্যার তৈরির প্র্যাকটিক্যাল মাস্টারক্লাস ও ডিব্যাগিং মাইন্ডসেট	পৃষ্ঠা ৩৬
অধ্যায় ১২	সিএস পরিভাষার পূর্ণাঙ্গ অভিধান (100+ Essential Terms Glossary)	পৃষ্ঠা ৩৯

CHAPTER 01

সফটওয়্যার কীভাবে কাজ করে? কম্পিউটারের অন্দরমহল ও ফান্ডামেন্টালস

কোড থেকে মেমোরি, প্রসেস, লিনাক্স শেল এবং এনভায়রনমেন্ট ভ্যারিয়েবলের গভীর বোঝাপড়া

১.১ কোড থেকে কারেন্ট: হাই-লেভেল ল্যাঙ্গুয়েজ বনাম মেশিন কোড

ইলেকট্রিক্যাল ইঞ্জিনিয়ার হিসেবে আপনি জানেন যে কম্পিউটারের একদম অন্তস্তলে সিলিকন চিপসে কোটি কোটি ট্রানজিস্টর রয়েছে। ট্রানজিস্টরে হাই ভোল্টেজ (সাধারণত ৩.৩V বা ৫V) থাকলে আমরা তাকে বলি লজিক '1', আর গ্রাউন্ড ভোল্টেজ (০V) থাকলে বলি '0'। কম্পিউটার মূলত এই ০ এবং ১ এর ডিজিটাল পালস ছাড়া সরাসরি আর কিছুই বোঝে না। একে বলা হয় মেশিন কোড (Machine Code) বা বাইনারি।

কিন্তু কোনো মানুষের পক্ষে বাইনারিতে কোড লেখা সম্ভব নয়। তাই মানুষ তৈরি করেছে বিভিন্ন স্তরের ল্যাঙ্গুয়েজ:

- মেশিন কোড (Machine Code): একদম কাঁচা বাইনারি নির্দেশাবলী (যেমন: `10110000 01100001`)।
- অ্যাসেম্বলি ল্যাঙ্গুয়েজ (Assembly): হার্ডওয়্যার রেজিস্টারের সরাসরি নির্দেশ (যেমন: `MOV EAX, 1`, `ADD EAX, EBX`)। মাইক্রোকন্ট্রোলারের ক্ষেত্রে আপনি এটি হয়তো দেখেছেন।
- হাই-লেভেল ল্যাঙ্গুয়েজ (High-Level Language): মানুষের মুখের ইংরেজি ভাষার মতো সিনট্যাক্স। যেমন: Python, JavaScript, TypeScript, C++, Rust, Go ইত্যাদি।

> ⚡ EEE অ্যানালজি: মাইক্রোকন্ট্রোলার ও ফার্মওয়্যার
>
> আপনি যখন অ্যারডুইনো (Arduino) বা STM32 মাইক্রোকন্ট্রোলারে C/C++ দিয়ে ফার্মওয়্যার লিখে 'Upload' বাটনে চাপেন, তখন পেছনের টুলচেইন সেই কোডটিকে হেক্স ফাইলে (.hex / .bin) রূপান্তর করে ফ্ল্যাশ মেমোরিতে লিখে দেয়। সফটওয়্যার জগতে পাইথন বা জাভাস্ক্রিপ্ট একই কাজ করে—শুধু মাঝে একটি শক্তিশালী অপারেটিং সিস্টেম (OS) থাকে যা হার্ডওয়্যার রিসোর্স পরিচালনা করে।

১.২ কম্পাইলার বনাম ইন্টারপ্রেটার: কোড কীভাবে রান করে?

হাই-লেভেল ভাষায় লেখা টেক্সট ফাইলকে কম্পিউটারের বোধগম্য মেশিন কোডে রূপান্তর করার মূলত দুটি পদ্ধতি রয়েছে:

বৈশিষ্ট্য	কম্পাইল্ড ল্যাঙ্গুয়েজ (Compiled)	ইন্টারপ্রেটেড ল্যাঙ্গুয়েজ (Interpreted)	হাইব্রিড / JIT (Just-In-Time)
উদাহরণ	C, C++, Go, Rust	Python, Ruby, Bash	JavaScript (V8 Engine), Java, C#
রূপান্তর পদ্ধতি	সম্পূর্ণ কোড একবারে মেশিন কোডে রূপান্তর করে একটি এক্সিকিউটেবল (.exe বা ELF বাইনারি) বানায়।	একটি প্রোগ্রাম (ইন্টারপ্রেটার) লাইন বাই লাইন কোড পড়ে এবং তাৎক্ষণিক রান করে।	প্রথমে বাইটকোডে (Bytecode) রূপান্তর করে, রানটাইমে দ্রুত মেশিন কোডে কম্পাইল করে।
গতির তুলনা	অত্যন্ত দ্রুত ও অপ্টিমাইজড।	তুলনামূলক ধীরগতির, তবে সহজে ডিবাগ করা যায়।	ভারসাম্যপূর্ণ এবং আধুনিক ওয়েব অ্যাপ্লিকেশনের মেরুদণ্ড।

> 🤖 এআই ব্যবহারের সময় কেন এটি জানা জরুরি?
>
> এআই যখন আপনাকে কোনো Go বা Rust প্রজেক্ট বানিয়ে দেয়, আপনাকে আগে `build` কমান্ড চালিয়ে বাইনারি তৈরি করতে হবে। আর পাইথন বা নোডজেএস (Node.js) প্রজেক্ট দিলে সরাসরি `python app.py` বা `node index.js` দিলেই রান হবে। এই পার্থক্যটি না বুঝলে টার্মিনালে এরর দেখে বিভ্রান্ত হতে হবে।

১.৩ মেমোরি আর্কিটেকচার: RAM, Stack vs Heap এবং মেমোরি লিক

যেকোনো সফটওয়্যার যখন রান করে, তার কোড এবং ব্যবহৃত সমস্ত ডেটা কম্পিউটারের র‍্যামে (RAM) জায়গা নেয়। মেমোরিকে মূলত দুটি অংশে ভাগ করা হয়:

- স্ট্যাক (Stack): অত্যন্ত দ্রুতগতির মেমোরি। ফাংশন কলের লোকাল ভ্যারিয়েবল ও এক্সিকিউশন ট্র্যাকিং এখানে সংরক্ষিত থাকে। ফাংশন শেষ হলে স্বয়ংক্রিয়ভাবে মেমোরি খালি হয়ে যায়।
- হিপ (Heap): ডায়নামিক ও বড় ডেটা রাখার মেমোরি। অবজেক্ট, বড় অ্যারে বা নেটওয়ার্ক রিকোয়েস্টের তথ্য এখানে জমা হয়। এখানে জায়গা বরাদ্দ ও মুক্ত করার জন্য ব্যাকগ্রাউন্ডে গার্বেজ কালেক্টর (Garbage Collector) কাজ করে।

> ⚠️ মেমোরি লিক (Memory Leak) কী?
>
> যদি কোনো সফটওয়্যার হিপ মেমোরিতে নতুন নতুন ডেটা রাখতে থাকে কিন্তু কাজ শেষে তা ডিলিট না করে, তবে সার্ভারের র‍্যাম একসময় ১০০% পূর্ণ হয়ে সার্ভার ক্র্যাশ করবে (Out of Memory - OOM Kill)। এআই অনেক সময় ইভেন্ট লিসেনার বা ডাটাবেজ কানেকশন ক্লোজ না করে রেখে দেয়—ফলে মেমোরি লিক তৈরি হয়।

১.৪ প্রসেস (Process), থ্রেড (Thread) ও ব্যাকগ্রাউন্ড ডেমন (Daemon)

সফটওয়্যার জগতে এই তিনটি শব্দ সবচেয়ে বেশি উচ্চারিত হয়:

- প্রসেস (Process): যখন একটি প্রোগ্রাম মেমোরিতে লোড হয়ে সিপিইউতে রান হতে শুরু করে, অপারেটিং সিস্টেম তাকে একটি নির্দিষ্ট প্রসেস আইডি (PID) এবং নিজস্ব সুরক্ষিত মেমোরি স্পেস বরাদ্দ করে। যেমন: আপনি টার্মিনালে `node server.js` রান করলে একটি নতুন প্রসেস তৈরি হয়।
- থ্রেড (Thread): একটি প্রসেসের ভেতরে একাধিক কাজ সমান্তরালে (Parallel) করার জন্য তৈরি হওয়া ছোট ছোট এক্সিকিউশন ইউনিট। যেমন: একটি ওয়েব ব্রাউজার একটি থ্রেডে ওয়েবপেজ রেন্ডার করে, আরেকটি থ্রেডে ব্যাকগ্রাউন্ডে ছবি ডাউনলোড করে।
- ডেমন / ব্যাকগ্রাউন্ড সার্ভিস (Daemon): যেসব সফটওয়্যার কোনো ইউজার ইন্টারফেস ছাড়া সার্ভারের ব্যাকগ্রাউন্ডে নিরবচ্ছিন্নভাবে চলতে থাকে (যেমন: ডাটাবেজ সার্ভার, ডকার ডেমন, Nginx)। লিনাক্সে এদের পরিচালনা করা হয় `systemctl` কমান্ড দিয়ে।

১.৫ এনভায়রনমেন্ট ভ্যারিয়েবল (Environment Variables / .env) কেন দরকার?

সফটওয়্যার ইঞ্জিনিয়ারিংয়ের সবচেয়ে গুরুত্বপূর্ণ নিরাপত্তা ও কনফিগারেশন নিয়ম হলো: "কখনই সোর্স কোডের ভেতরে পাসওয়ার্ড, ডাটাবেজ ইউআরএল বা এপিআই কি সরাসরি লিখে রাখবেন না (Hardcoding is a crime!)"।

এর পরিবর্তে ব্যবহার করা হয় এনভায়রনমেন্ট ভ্যারিয়েবল। এটি একটি সাধারণ কি-ভ্যালু পেয়ার যা অপারেটিং সিস্টেমের পরিবেশে সংরক্ষিত থাকে।

```
# .env ফাইলের নমুনা
DATABASE_URL=postgresql://postgres:password123@localhost:5432/mydb
OPENAI_API_KEY=sk-proj-abc123xyz789
PORT=3000
NODE_ENV=production
```

সফটওয়্যার রান হওয়ার সময় কোডটি বলে: "আমাকে সিস্টেমের `OPENAI_API_KEY` এর মান এনে দাও"। ফলে কোড যদি আপনি গিটহাবে পাবলিকলি আপলোডও করেন, আপনার গোপনীয় চাবি সুরক্ষিত থাকবে।

১.৬ টার্মিনাল ও লিনাক্স সিএলআই (CLI) মাস্টারি

আধুনিক ক্লাউড, ডকার ও সার্ভারের ৯০%+ সিস্টেম চলে লিনাক্সে (Linux)। একজন নন-সিএস ইঞ্জিনিয়ার হিসেবে মাউসের বদলে কীবোর্ড দিয়ে টার্মিনাল নিয়ন্ত্রণ করার মৌলিক কমান্ডগুলো আয়ত্ত করা জরুরি:

কমান্ড	কাজ	বাস্তব উদাহরণ
`pwd` & `ls -la`	বর্তমান ফোল্ডার দেখা ও ফাইলের তালিকা (হিডেন ফাইলসহ) দেখা	`ls -lh` দিয়ে ফাইলের সাইজ দেখা
`cd <dir>`	ফোল্ডার পরিবর্তন করা (Change Directory)	`cd ..` (এক স্তর ব্যাকে যাওয়া)
`mkdir <name>`	নতুন ফোল্ডার তৈরি করা	`mkdir -p src/controllers`
`cat <file>` / `nano`	ফাইলের কনটেন্ট পড়া বা টার্মিনালে এডিট করা	`nano .env`
`ps aux | grep node`	চলমান নোডজেএস প্রসেসের PID খুঁজে বের করা	কোন অ্যাপ চলছে তা দেখা
`kill -9 <PID>`	অবাধ্য বা আটকে যাওয়া কোনো প্রসেসকে জোরপূর্বক বন্ধ করা	পোর্ট খালি করার জন্য ব্যবহৃত হয়
`chmod` & `chown`	ফাইলের পারমিশন ও ওনারশিপ পরিবর্তন করা	`chmod +x script.sh` (এক্সিকিউট পারমিশন)

📌 অধ্যায় ১ এর সারসংক্ষেপ

- কম্পাইলার কোডকে মেশিন কোড বাইনারিতে রূপান্তর করে, ইন্টারপ্রেটার রানটাইমে লাইন বাই লাইন চালায়।
- RAM এর স্ট্যাক মেমোরি দ্রুত ও অস্থায়ী, হিপ মেমোরি ডায়নামিক অবজেক্টের জন্য। মেমোরি লিক হলে সার্ভার ক্র্যাশ করে।
- প্রসেস হলো মেমোরিতে জীবন্ত সফটওয়্যার, থ্রেড হলো তার ভেতরের প্যারালাল কর্মী।
- এনভায়রনমেন্ট ভ্যারিয়েবল (.env) দিয়ে কনফিগারেশন ও পাসওয়ার্ড কোড থেকে আলাদা রাখা হয়।
- লিনাক্স টার্মিনাল হলো সার্ভার ও ক্লাউডের মূল ড্রাইভিং সিট।

CHAPTER 02

আধুনিক সফটওয়্যার আর্কিটেকচার: ওয়েব ও মোবাইল অ্যাপের কাঠামো

ক্লায়েন্ট-সার্ভার মডেল, ফ্রন্টএন্ড বনাম ব্যাকএন্ড, মোবাইল টেকনোলজি এবং রেন্ডারিং মেথডোলজি

২.১ ক্লায়েন্ট-সার্ভার মডেল (Client-Server Architecture)

বিশ্বের প্রায় প্রতিটি আধুনিক ইন্টারনেট-ভিত্তিক সফটওয়্যার (ফেসবুক, ইউটিউব, উবার, চ্যাটজিপিটি) ক্লায়েন্ট-সার্ভার মডেলে কাজ করে।

> 💡 বাস্তব জীবনের অ্যানালজি: রেস্তোরাঁ মডেল
>
> একটি রেস্তোরাঁর কথা চিন্তা করুন:
> ১. ক্লায়েন্ট (Client): আপনি (গ্রাহক) চেয়ার-টেবিলে বসে আছেন। আপনার সামনে একটি সুন্দর মেন্যু কার্ড আছে।
> ২. এপিআই ও নেটওয়ার্ক (API / Network): ওয়েটার, যিনি আপনার কাছ থেকে অর্ডার নিয়ে রান্নাঘরে যান এবং খাবার নিয়ে ফিরে আসেন।
> ৩. সার্ভার (Server): রেস্তোরাঁর সুরক্ষিত রান্নাঘর, যেখানে শেফ (বিজনেস লজিক) খাবার রান্না করেন।
> ৪. ডাটাবেজ (Database): পেছনের স্টোররুম বা ডিপ ফ্রিজ, যেখান থেকে শেফ প্রয়োজনীয় কাঁচামাল সংগ্রহ করেন।

গ্রাহক কখনোই রান্নাঘরে ঢুকতে পারেন না; ঠিক তেমনি একজন সাধারণ ইউজার ব্রাউজার বা মোবাইল ফোন (ক্লায়েন্ট) দিয়ে সরাসরি ডাটাবেজ স্পর্শ করতে পারেন না। মাঝখানে সার্ভার নিরাপত্তা ও লজিকের প্রহরী হিসেবে কাজ করে।

২.২ ফ্রন্টএন্ড (Frontend): ইন্টারফেসের জাদুকরী দুনিয়া

ফ্রন্টএন্ড হলো সফটওয়্যারের সেই অংশ যা ইউজার তার চোখে দেখতে পায় এবং যার সাথে মাউস বা টাচ দিয়ে ইন্টার‍্যাক্ট করে। ব্রাউজারে মূলত তিনটি স্তম্ভ কাজ করে:

- HTML (HyperText Markup Language): কাঠ বা কঙ্কাল। টেক্সট, বাটন, ইনপুট ফিল্ড ও ইমেজ ডিফাইন করে।
- CSS (Cascading Style Sheets): রং, ডিজাইন ও লেআউট। বাটনটি দেখতে নীল হবে নাকি লাল, ড্রপডাউন অ্যানিমেশন কেমন হবে তা নির্ধারণ করে।
- JavaScript / TypeScript: প্রাণশক্তি বা স্নায়ুতন্ত্র। বাটনে ক্লিক করলে কী ঘটবে, ফর্ম ভ্যালিডেশন এবং সার্ভারে রিকোয়েস্ট পাঠানো জাভাস্ক্রিপ্টের কাজ।

> ⚡ EEE অ্যানালজি: ইলেকট্রনিক্স প্রজেক্টের সাথে তুলনা
>
> একটি ডিজিটাল মাল্টিমিটারের কথা ভাবুন। মাল্টিমিটারের প্লাস্টিক কেসিং এবং ডিসপ্লে স্ক্রিন হলো Frontend (HTML/CSS), সিলেক্টর নব এবং বাটন হলো UI Components, ভেতরের ADC ও মাইক্রোকন্ট্রোলার যা সিগন্যাল মাপছে তা হলো Backend, আর ভেতরের EEPROM চিপ যেখানে রিডিং সেভ থাকে তা হলো Database!

আধুনিক ফ্রন্টএন্ড ফ্রেমওয়ার্ক: কেন সাধারণ HTML/JS যথেষ্ট নয়?

আজকের ওয়েব অ্যাপগুলোতে (যেমন জিমেইল বা ফেসবুক) প্রতি মুহূর্তে শত শত ডেটা আপডেট হয়। বারবার পুরো পেজ রিলোড করা অত্যন্ত ধীরগতির। তাই জন্ম নিয়েছে কম্পোনেন্ট-বেসড ফ্রেমওয়ার্ক:

- React.js: মেটার তৈরি বিশ্বের সবচেয়ে জনপ্রিয় UI লাইব্রেরি। এটি পেজকে ছোট ছোট ব্লকে (Component) ভাগ করে এবং Virtual DOM ব্যবহার করে শুধু পরিবর্তিত অংশটুকু রি-রেন্ডার করে।
- Next.js: রিঅ্যাক্টের ওপর তৈরি অল-ইন-ওয়ান ফ্রেমওয়ার্ক। এটি সার্ভার-সাইড রেন্ডারিং এবং এসইও (SEO) ফ্রেন্ডলি পেজ তৈরির জন্য ইন্ডাস্ট্রি স্ট্যান্ডার্ড।
- Vue.js & Svelte: দ্রুত শেখার উপযোগী এবং অত্যন্ত মসৃণ পারফরম্যান্সের লাইব্রেরি।

২.৩ ব্যাকএন্ড (Backend): সফটওয়্যারের মস্তিষ্ক ও ইঞ্জিন

ব্যাকএন্ড হলো ইন্টারনেটের পেছনে চলা শক্তিশালী সার্ভার প্রোগ্রাম। এর প্রধান দায়িত্ব চারটি:

1. বিজনেস লজিক এক্সিকিউশন: যেমন "ইউজার যদি ১০০০ টাকার শপিং করে তবে ১০% ডিসকাউন্ট যোগ করো"।
2. নিরাপত্তা ও অথেন্টিকেশন: পাসওয়ার্ড যাচাই করা এবং পাসওয়ার্ড হ্যাশ করে রাখা (bcrypt)।
3. ডাটাবেজের সাথে যোগাযোগ: নিরাপদভাবে ডেটা সেভ করা ও কুয়েরি করা।
4. অন্যান্য সার্ভিসের সাথে যোগাযোগ: পেমেন্ট গেটওয়ে (Stripe/bKash), ইমেইল পাঠানো বা এসএমএস পাঠানো।

জনপ্রিয় ব্যাকএন্ড টেকনোলজি স্ট্যাক

ল্যাঙ্গুয়েজ	ফ্রেমওয়ার্ক	শক্তি ও ব্যবহারের ক্ষেত্র
JavaScript / TS	Express.js, Nest.js, Fastify	একই ভাষায় ফ্রন্টএন্ড ও ব্যাকএন্ড লেখা যায় (Fullstack JS)। দ্রুত প্রোটোটাইপিংয়ের সেরা।
Python	FastAPI, Django, Flask	এআই, মেশিন লার্নিং ও ডাটা সায়েন্স প্রজেক্টের ব্যাকএন্ডের অবিসংবাদিত রাজা।
Go (Golang)	Gin, Echo, Fiber	অত্যন্ত দ্রুত, কম মেমোরি খরচ এবং হাই-স্কেল মাইক্রোসার্ভিসের জন্য আদর্শ।

২.৪ মোবাইল অ্যাপ আর্কিটেকচার: Native বনাম Cross-Platform

আপনি যদি এআই দিয়ে একটি অ্যান্ড্রয়েড বা আইওএস (iOS) মোবাইল অ্যাপ বানাতে চান, আপনার সামনে দুটি পথ রয়েছে:

- নেটিভ ডেভেলপমেন্ট (Native): অ্যান্ড্রয়েডের জন্য Kotlin/Java এবং অ্যাপলের জন্য Swift/Objective-C দিয়ে কোড লেখা। সর্বোচ্চ পারফরম্যান্স পাওয়া যায়, তবে দুটি আলাদা কোডবেস ম্যানেজ করতে হয়।
- ক্রস-প্ল্যাটফর্ম ডেভেলপমেন্ট (Cross-Platform): একবার কোড লিখে অ্যান্ড্রয়েড এবং আইওএস উভয় প্ল্যাটফর্মেই অ্যাপ চালানো যায়।
  - Flutter (Dart): গুগলের তৈরি। সরাসরি ক্যানভাসে পিক্সেল আঁকে, ফলে অসাধারণ মসৃণ UI পাওয়া যায়।
  - React Native (JS/TS): ফেসবুকের তৈরি। রিঅ্যাক্টের নলেজ দিয়েই মোবাইল অ্যাপ বানানো যায়।

> 🤖 এআই দিয়ে প্রথম অ্যাপ তৈরির সেরা রিকমেন্ডেশন
>
> শুরুতে মোবাইল অ্যাপ তৈরি করতে চাইলে Flutter অথবা React Native (Expo) বেছে নিন। এআই (যেমন Cursor বা Claude) এক্সপো দিয়ে অবিশ্বাস্য দ্রুত অ্যান্ড্রয়েড ও আইওএস অ্যাপ তৈরি করে দিতে পারে।

২.৫ CSR, SSR, SSG এবং ISR এর সহজ হিসাব

এআই-এর সাথে কাজ করার সময় আপনি প্রায়ই Next.js বা আধুনিক ওয়েব ডেভেলপমেন্টে এই চারটি শব্দ শুনবেন:

পদ্ধতি	অর্থ	কীভাবে কাজ করে?	কখন ব্যবহার করবেন?
CSR	Client-Side Rendering	ব্রাউজার কেবল ফাঁকা HTML পায়, তারপর জাভাস্ক্রিপ্ট দিয়ে ক্লায়েন্ট ডিভাইসে পুরো UI তৈরি হয়।	ড্যাশবোর্ড, অ্যাডমিন প্যানেল যেখানে এসইও জরুরি নয়।
SSR	Server-Side Rendering	ইউজার প্রতিবার পেজ চাইলে সার্ভার তাৎক্ষণিক ডাটাবেজ থেকে ডেটা নিয়ে সম্পূর্ণ HTML তৈরি করে ব্রাউজারে পাঠায়।	ই-কমার্স প্রোডাক্ট পেজ, যেখানে রিয়েল-টাইম প্রাইস ও এসইও দুটোই দরকার।
SSG	Static Site Generation	বিল্ড করার সময়েই সমস্ত পেজের HTML তৈরি হয়ে যায়। সার্ভার শুধু ফাইল ডেলিভার করে।	ব্লগ, ডকুমেন্টেশন, পোর্টফোলিও ওয়েবসাইট (সুপার ফাস্ট)।

📌 অধ্যায় ২ এর সারসংক্ষেপ

- ক্লায়েন্ট হলো ইউজার ইন্টারফেস (ব্রাউজার/অ্যাপ), সার্ভার হলো নিরাপদ বিজনেস লজিক ও ডেটা ম্যানেজার।
- ফ্রন্টএন্ডের মূল তিনটি স্তম্ভ: HTML (স্ট্রাকচার), CSS (স্টাইল), JS (ইন্টার‍্যাকশন)। আধুনিক বিশ্বে রিঅ্যাক্ট/নেক্সটজেএস শীর্ষে।
- ব্যাকএন্ডে নোডজেএস, পাইথন (FastAPI) ও গো বহুল ব্যবহৃত।
- মোবাইল অ্যাপে একই কোডবেসে দুই প্ল্যাটফর্মের জন্য Flutter ও React Native সবচেয়ে উপযোগী।
- বিল্ড টাইমেই পেজ বানিয়ে রাখলে SSG, প্রতি রিকোয়েস্টে সার্ভারে বানালে SSR, আর ব্রাউজারে বানালে CSR।

CHAPTER 03

নেটওয়ার্কিং, ইন্টারনেট ও এপিআই (APIs) এর নাড়িনক্ষত্র

আইপি, ডিএনএস, এইচটিটিপি মেথডস, স্ট্যাটাস কোড, জেএসন, কর্স (CORS) ও আধুনিক অথেনটিকেশন

৩.১ ইন্টারনেট কীভাবে কাজ করে: IP, DNS, Domain এবং Ports

ইন্টারনেট আসলে কোটি কোটি কম্পিউটারের একটি বিশাল অপটিক্যাল ফাইবার ও তারের জালিকা। ইন্টারনেটে প্রতিটি ডিভাইসের একটি অনন্য পরিচয় থাকে যাকে বলা হয় IP Address (Internet Protocol) (যেমন: `142.250.190.46`)।

- DNS (Domain Name System): মানুষের পক্ষে সংখ্যা মনে রাখা কঠিন। তাই ব্রাউজারে `google.com` লিখলে ডিএনএস সার্ভার ডিরেক্টরি ঘেঁটে গুগলের আসল আইপি অ্যাড্রেস এনে দেয়। এটি ইন্টারনেটের ফোনবুক।
- Port (পোর্ট): একটি কম্পিউটারে বা সার্ভারে একই সাথে অনেক সফটওয়্যার চলতে পারে। পোর্ট হলো সার্ভারের ভার্চুয়াল দরজা।
  - `80` : সাধারণ আনসিকিউরড HTTP ওয়েব ট্রাফিকের জন্য।
  - `443` : নিরাপদ এনক্রিপ্টেড HTTPS ট্রাফিকের জন্য।
  - `3000` / `5173` / `8080` : লোকাল ডেভেলপমেন্টে রিঅ্যাক্ট বা নোড সার্ভার চালানোর জন্য প্রচলিত পোর্ট।
  - `5432` : PostgreSQL ডাটাবেজের ডিফল্ট পোর্ট।

> ⚡ EEE অ্যানালজি: আইপি অ্যাড্রেস ও পোর্ট বনাম হার্ডওয়্যার অ্যাড্রেসিং
>
> মাইক্রোকন্ট্রোলারে যখন একাধিক সেন্সর I2C বাস (I2C Bus) দিয়ে যুক্ত থাকে, তখন মাস্টার চিপটি প্রতি সেন্সরের ৭-বিট I2C অ্যাড্রেস (Address) দিয়ে তাদের সাথে কথা বলে। কম্পিউটারের আইপি অ্যাড্রেস হলো সেই নির্দিষ্ট চিপের অ্যাড্রেস, আর পোর্ট হলো চিপের ভেতরের নির্দিষ্ট রেজিস্টার বা পিন নম্বর যার সাথে নির্দিষ্ট অ্যাপ্লিকেশনটি সংযুক্ত!

৩.২ HTTP ও HTTPS: ইন্টারনেটের ভাষা

ব্রাউজার এবং সার্ভারের মধ্যকার সব যোগাযোগ হয় HTTP (HyperText Transfer Protocol) নিয়মে। এটি একটি রিকোয়েস্ট-রেসপন্স সাইকেল:

1. ক্লায়েন্ট পাঠায় একটি HTTP Request (URL, Method, Headers, Body)।
2. সার্ভার প্রসেস করে ফেরত পাঠায় একটি HTTP Response (Status Code, Headers, Data)।

এইচটিটিপি স্ট্যাটাস কোড (HTTP Status Codes) যা আপনার মুখস্থ থাকা উচিত

রেঞ্জ	বিভাগ	গুরুত্বপূর্ণ কোডসমূহ ও তাদের অর্থ
`2xx`	Success	200 OK: সব ঠিকঠাক আছে। 201 Created: নতুন ডেটা সফলভাবে তৈরি হয়েছে।
`3xx`	Redirection	301 / 302: অন্য পেজে রিডাইরেক্ট করা হয়েছে।
`4xx`	Client Error (ক্লায়েন্টের ভুল)	400 Bad Request: ভুল ডেটা পাঠানো হয়েছে। 401 Unauthorized: লগইন করা নেই (নো টোকেন)। 403 Forbidden: অ্যাক্সেস পারমিশন নেই। 404 Not Found: এই ইউআরএল বা পেজের অস্তিত্ব নেই।
`5xx`	Server Error (সার্ভারের সমস্যা)	500 Internal Server Error: ব্যাকএন্ড কোড ক্র্যাশ করেছে। 502 Bad Gateway / 504 Gateway Timeout: প্রক্সি সার্ভার মূল ব্যাকএন্ডের সাড়া পাচ্ছে না।

৩.৩ এপিআই (API) এবং RESTful আর্কিটেকচার

API মানে Application Programming Interface। এটি দুটি সফটওয়্যারের একে অপরের সাথে কথা বলার মাধ্যম। আধুনিক ওয়েবে বহুল ব্যবহৃত হলো REST API। এতে প্রধানত ৪টি মেথড (CRUD) থাকে:

- `GET /api/users` : সমস্ত ইউজারের তথ্য পড়ুন (Read)।
- `POST /api/users` : নতুন ইউজার তৈরি করুন (Create)। ডেটা পাঠানো হয় Request Body-তে।
- `PUT / PATCH /api/users/12` : ইউজার ১২ এর তথ্য আপডেট করুন (Update)।
- `DELETE /api/users/12` : ইউজার ১২ কে মুছে ফেলুন (Delete)।

৩.৪ রিয়েল-টাইম কমিউনিকেশন: REST বনাম WebSockets

সাধারণ REST API-তে ক্লায়েন্ট না চাইলে সার্ভার নিজে থেকে কিছু পাঠাতে পারে না (হ্যান্ডশেক অ্যান্ড ক্লোজ)। কিন্তু আপনি যদি একটি লাইভ চ্যাট অ্যাপ (যেমন WhatsApp) বা ক্রিপ্টোকারেন্সির লাইভ প্রাইস ট্র্যাকার বানাতে চান?

> 💡 ফোন কল বনাম চিঠি আদান-প্রদান
>
> REST API হলো চিঠি পাঠানোর মতো—আপনি চিঠি পাঠাবেন, ডাকপিয়ন উত্তর নিয়ে আসবে এবং সংযোগ শেষ। আর WebSocket হলো একটি চালু টেলিফোন কলের মতো—উভয় প্রান্ত সার্বক্ষণিক একটি দীর্ঘমেয়াদী বাই-ডিরেকশনাল সকেট সংযোগে উন্মুক্ত থাকে। সার্ভার যেকোনো মুহূর্তে চোখের পলকে ক্লায়েন্টে নতুন মেসেজ ঠেলে (Push) দিতে পারে।

৩.৫ JSON (JavaScript Object Notation): সফটওয়্যারের বিশ্বভাষা

আজকের দুনিয়ায় পাইথন, রিঅ্যাক্ট, ডকার বা চ্যাটজিপিটি সবাই তথ্য আদান-প্রদান করে JSON ফরম্যাটে। এটি মানুষের পড়ার উপযোগী এবং হালকা টেক্সট ডেটা ফরম্যাট:

```
{
  "project": "AI Smart Assistant",
  "version": 1.0,
  "isActive": true,
  "tags": ["embedded", "devops", "cloud"],
  "developer": {
    "name": "Engineer Kabir",
    "degree": "EEE"
  }
}
```

৩.৬ CORS এরর: ডেভেলপারদের চিরন্তন আতঙ্ক ও সমাধান

এআই দিয়ে ফ্রন্টএন্ড (React runs on `http://localhost:5173`) এবং ব্যাকএন্ড (Express/FastAPI runs on `http://localhost:5000`) বানানোর পর ব্রাউজার কনসোলে বড় লাল কালিতে লেখা আসে:

```
Access to fetch at 'http://localhost:5000/api' from origin 'http://localhost:5173'
has been blocked by CORS policy: No 'Access-Control-Allow-Origin' header is present!
```

কেন এই এরর আসে? ব্রাউজারের নিরাপত্তার স্বার্থে Same-Origin Policy রয়েছে। পোর্ট 5173 এবং পোর্ট 5000 ব্রাউজারের চোখে দুটি ভিন্ন উৎস (Origin)। তাই ব্যাকএন্ড সার্ভার যদি সুস্পষ্টভাবে বলে না দেয় যে "হ্যাঁ, পোর্ট 5173 থেকে রিকোয়েস্ট আমি গ্রহণ করব", ব্রাউজার সেই ডেটা ব্লক করে দেবে।

> 🤖 CORS সমস্যা সমাধানের ম্যাজিক এআই প্রম্পট
>
> ব্যাকএন্ডে `cors` মিডলওয়্যার যুক্ত করতে এআই-কে বলুন:
> "My frontend runs on http://localhost:5173. Please configure CORS middleware in my FastAPI/Express backend to allow requests, credentials, and headers from this origin."

৩.৭ অথেনটিকেশন ও সিকিউরিটি: Session বনাম JWT

ইউজার লগইন করলে সার্ভার কীভাবে মনে রাখে যে তিনি আসলেই অথেনটিকেটেড ইউজার?

- Session & Cookies: পুরনো কিন্তু অত্যন্ত নিরাপদ মডেল। ইউজার লগইন করলে সার্ভারের মেমোরিতে একটি সেশন আইডি তৈরি হয় এবং ব্রাউজারে একটি সুরক্ষিত কুকি (HttpOnly Cookie) পাঠানো হয়।
- JWT (JSON Web Token): আধুনিক স্ট্যাটলেস (Stateless) মডেল। লগইন করলে সার্ভার একটি ডিজিটাল ক্রিপ্টোগ্রাফিক সইযুক্ত টোকেন বানিয়ে ক্লায়েন্টকে দেয়। ক্লায়েন্ট পরবর্তীতে প্রতি রিকোয়েস্টের হেডারে পাঠায়:
  `Authorization: Bearer <JWT_TOKEN_STRING>`
  সার্ভার কোনো ডাটাবেজ লুকআপ ছাড়াই শুধু নিজের প্রাইভেট কি দিয়ে টোকেনের সত্যতা যাচাই করে নিতে পারে।
- OAuth 2.0: "Sign in with Google" বা "Login with GitHub" এর আন্তর্জাতিক সিকিউরড স্ট্যান্ডার্ড প্রোটোকল। এতে ইউজারকে আপনার অ্যাপে পাসওয়ার্ড টাইপ করতে হয় না।

📌 অধ্যায় ৩ এর সারসংক্ষেপ

- আইপি হলো সার্ভারের ঠিকানা, পোর্ট হলো সফটওয়্যারের ভার্চুয়াল দরজা (HTTP: 80, HTTPS: 443)।
- HTTP মেথডস: GET (পড়া), POST (তৈরি), PUT (আপডেট), DELETE (মুছে ফেলা)।
- স্ট্যাটাস কোড: 200 মানে সাফল্য, 401 মানে অথেন্টিকেশন দরকার, 404 মানে খুঁজে পাওয়া যায়নি, 500 মানে সার্ভার ক্র্যাশ।
- লাইভ রিয়েল-টাইম দুইমুখী যোগাযোগের জন্য WebSockets অপরিহার্য।
- CORS সমস্যা সমাধান করতে ব্যাকএন্ডে অরিজিন হোয়াইটলিস্ট বা cors মিডলওয়্যার কনফিগার করতে হয়।
- JWT টোকেন দিয়ে আধুনিক ক্লাউড ও মোবাইল অ্যাপের স্টেটলেস অথেনটিকেশন পরিচালিত হয়।

CHAPTER 04

ডেটাবেজ ও স্টোরেজ ম্যানেজমেন্ট: তথ্য সংরক্ষণের কলাকৌশল

SQL বনাম NoSQL, অবজেক্ট স্টোরেজ, ওআরএম (ORM), মাইগ্রেশন এবং আধুনিক ক্লাউড ডাটাবেজ

৪.১ ডেটাবেজ কেন লাগে? ভলাটাইল মেমোরি বনাম পারসিস্টেন্ট স্টোরেজ

আপনার সফটওয়্যারের ভ্যারিয়েবলে রাখা যেকোনো ডেটা কেবল কম্পিউটারের র‍্যামে (RAM) অবস্থান করে। বিদ্যুৎ চলে গেলে কিংবা সার্ভার একবার রিস্টার্ট নিলে র‍্যামের সমস্ত তথ্য চিরতরে মুছে যায় (Volatile)। তাই হাজার হাজার ইউজারের অ্যাকাউন্ট ব্যালেন্স, পাসওয়ার্ড ও অর্ডার হিস্ট্রি নিরাপদে হার্ডডিস্কে দীর্ঘমেয়াদে সংরক্ষণ ও বিদ্যুৎগতির সার্চ করার জন্য বিশেষভাবে তৈরি সফটওয়্যারকে বলা হয় Database Management System (DBMS)।

> ⚡ EEE অ্যানালজি: SRAM বনাম EEPROM বনাম SD Card
>
> মাইক্রোকন্ট্রোলারের ভেতরের SRAM হলো সফটওয়্যারের লোকাল ভ্যারিয়েবল (পাওয়ার কাটলে শূন্য)। ভেতরের EEPROM বা ফ্ল্যাশ মেমোরি হলো ডেটাবেজ যেখানে কনফিগারেশন সংরক্ষিত থাকে। আর প্রজেক্টে যুক্ত করা বড় মেমোরি কার্ড বা এক্সটার্নাল SD Card হলো Object Storage (AWS S3) যেখানে বড় বড় লগ ফাইল বা অডিও-ভিডিও সংরক্ষিত থাকে!

৪.২ রিলেশনাল ডাটাবেজ (SQL): কাঠামোগত ডেটার স্বর্গ

রিলেশনাল ডাটাবেজে তথ্য সংরক্ষিত থাকে এক্সেল শিটের মতো নিখুঁত সারি (Row) এবং কলাম (Column) আকারে। এই টেবিলগুলোকে বলা হয় Schema। যেমন: `users` টেবিল, `orders` টেবিল।

- Primary Key (PK): একটি রো-এর অনন্য পরিচয় (যেমন `id = 101`)।
- Foreign Key (FK): অন্য টেবিলের সাথে সম্পর্ক তৈরি করার চাবি। যেমন `orders` টেবিলে `user_id` কলাম যা `users` টেবিলের আইডির সাথে লিংকড।
- ACID প্রোপার্টিজ: Atomicity, Consistency, Isolation, Durability। ব্যাংকিং বা ফিনান্সিয়াল ট্রানজ্যাকশনে এসকিউএল ডাটাবেজের বিকল্প নেই। কারণ মাঝপথে টাকা কেটে নিয়ে অন্য অ্যাকাউন্টে যোগ না হলে পুরো ট্রানজ্যাকশন স্বয়ংক্রিয়ভাবে রোলব্যাক হয়।
- জনপ্রিয় ইঞ্জিন: PostgreSQL (আধুনিক বিশ্বের সেরা ওপেন-সোর্স এসকিউএল), MySQL, SQLite (একটি সিঙ্গেল ফাইলে চলা হালকা ডাটাবেজ)।

৪.৩ নন-রিলেশনাল ডাটাবেজ (NoSQL): স্কিমাহীন নমনীয়তা

সব ডেটার কাঠামো সবসময় একরকম হয় না। সামাজিক মাধ্যমের পোস্ট, কমেন্ট, কিংবা জটিল আইওটি (IoT) সেন্সরের ডেটার জন্য টেবিলের বদলে JSON-এর মতো ডকুমেন্টে তথ্য সংরক্ষণ করা অনেক সহজ। একে বলে NoSQL।

ধরন	জনপ্রিয় ডাটাবেজ	কাজের ক্ষেত্র ও সুবিধা
Document Store	MongoDB, CouchDB	প্রতিটি রেকর্ড একটি JSON ডকুমেন্ট। কোনো ফিক্সড স্কিমা নেই, দ্রুত প্রোটোটাইপিংয়ের জন্য চমৎকার।
Key-Value Store / Cache	Redis, Memcached	র‍্যামে ডেটা সংরক্ষণ করে, মাইক্রোসেকেন্ডে রেসপন্স দেয়। সেশন ক্যাশ, ওটিপি (OTP) ও রেট লিমিটিংয়ে ব্যবহৃত হয়।
Graph Database	Neo4j	সোশ্যাল নেটওয়ার্কের ফ্রেন্ড রিকমেন্ডেশন বা নলেজ গ্রাফের গভীর রিলেশন দ্রুত খোঁজার জন্য।

৪.৪ অবজেক্ট স্টোরেজ (Object Storage): ইমেজ ও পিডিএফ কোথায় রাখবেন?

> ⚠️ বিগিনারদের মারাত্মক ভুল: ডাটাবেজে ফাইল সংরক্ষণ
>
> কখনই ইউজারের প্রোফাইল পিকচার, পিডিএফ ফাইল বা ভিডিও সরাসরি এসকিউএল ডাটাবেজে BLOB হিসেবে সংরক্ষণ করবেন না! এতে ডাটাবেজের সাইজ গিগাবাইটে পৌঁছাবে, ব্যাকআপ নেওয়া অসম্ভব হবে এবং কুয়েরি ধীরগতির হয়ে যাবে।

সঠিক নিয়ম: ছবি বা ফাইলটি আপলোড করবেন AWS S3, Cloudflare R2 অথবা Google Cloud Storage-এ। স্টোরেজ আপনাকে একটি পাবলিক ইউআরএল (যেমন: `https://cdn.mysite.com/images/avatar12.jpg`) দেবে। ডাটাবেজে কেবল সেই ইউআরএল স্ট্রিংটি সেভ করে রাখবেন!

৪.৫ ওআরএম (ORM): কোড দিয়ে ডাটাবেজ জয়

আগে ডেভেলপারদের কাঁচা এসকিউএল কুয়েরি লিখতে হতো:
`SELECT * FROM users WHERE email = 'engr@gmail.com' AND is_active = true;`

ORM (Object-Relational Mapping) আপনাকে আপনার প্রিয় প্রোগ্রামিং ল্যাঙ্গুয়েজের সাধারণ অবজেক্ট বা মেথড দিয়ে ডাটাবেজ নিয়ন্ত্রণ করতে দেয়:

```
// Prisma ORM (TypeScript) দিয়ে ডাটাবেজ কুয়েরি
const user = await prisma.user.findUnique({
  where: { email: 'engr@gmail.com' },
  include: { orders: true }
});
```

জনপ্রিয় ওআরএম টুলস: Prisma ও Drizzle (Node.js/TypeScript-এ সর্বাধিক জনপ্রিয়), SQLAlchemy (পাইথনের স্ট্যান্ডার্ড), Django ORM।

৪.৬ ডাটাবেজ মাইগ্রেশন (Database Migration) কি?

ধরা যাক, আপনার অ্যাপ প্রোডাকশনে লাইভ চলছে এবং সেখানে ১০,০০০ ইউজার ডেটা রয়েছে। এখন আপনি নতুন ফিচার যোগ করার জন্য `users` টেবিলে `phone_number` নামে একটি নতুন কলাম যোগ করতে চান।

সরাসরি প্রোডাকশন ডাটাবেজে ম্যানুয়ালি ক্লিক করে এডিট করলে সিস্টেমে অসঙ্গতি তৈরি হতে পারে। ডাটাবেজ মাইগ্রেশন হলো ডাটাবেজ স্কিমার জন্য একটি ভার্সন কন্ট্রোল সিস্টেম। এটি কোডের মতো ধাপে ধাপে রেকর্ড রাখে যে কোন তারিখে টেবিলে কী কী পরিবর্তন আনা

হয়েছে (যেমন: `001_create_users.sql`, `002_add_phone_column.sql`)। ফলে নতুন কোনো সার্ভার বসালে এক ক্লিকে পুরো ডাটাবেজ কাঠামো তৈরি হয়ে যায়।

৪.৭ আধুনিক ক্লাউড BaaS: Supabase ও Firebase

আপনি যদি এআই দিয়ে খুব দ্রুত একটি প্রজেক্ট দাঁড় করাতে চান এবং ডাটাবেজ ও ব্যাকএন্ডের পেছনে বেশি সময় নষ্ট না করতে চান, তবে BaaS (Backend-as-a-Service) ব্যবহার করুন:

- Supabase (Open Source Firebase Alternative): এটি সম্পূর্ণ ম্যানেজড PostgreSQL ডাটাবেজ দেয়। একই সাথে ইউজার লগইন (Auth), ফাইল স্টোরেজ (S3) এবং অটো-জেনারেটেড REST/GraphQL API ফ্রিতে দিয়ে দেয়। এআই-এর সাথে কাজ করার জন্য এটি বর্তমানে সবচেয়ে শক্তিশালী স্ট্যাক।
- Firebase (Google): গুগলের রিয়েল-টাইম NoSQL ডাটাবেজ (Firestore) ও ক্লাউড নোটিফিকেশন সিস্টেম। মোবাইল অ্যাপ তৈরিতে ব্যাপক জনপ্রিয়।

📌 অধ্যায় ৪ এর সারসংক্ষেপ

- এসকিউএল (PostgreSQL) ফিক্সড স্কিমা ও কঠোর ACID নিয়মে চলে—ফিনান্সিয়াল ও কোর অ্যাপের মূল চাবিকাঠি।
- NoSQL (MongoDB) দ্রুত ফ্লেক্সিবল ডেটা হ্যান্ডল করে; Redis ব্যবহার হয় আল্ট্রা-ফাস্ট ক্যাশিং ও ওটিপির জন্য।
- ফাইল/ইমেজ কখনোই ডাটাবেজে নয়, যাবে অবজেক্ট স্টোরেজে (AWS S3/Cloudflare R2); ডাটাবেজে শুধু থাকবে লিংক।
- ORM (যেমন Prisma/SQLAlchemy) কাঁচা এসকিউএল না লিখে অবজেক্ট মেথড দিয়ে নিরাপদে ডাটাবেজ কুয়েরি করতে দেয়।
- ডাটাবেজ মাইগ্রেশন হলো স্কিমার ইতিহাস ও কোড ভার্সন কন্ট্রোল।
- সুপাবেস (Supabase) এআই বিল্ডারদের জন্য ব্যাকএন্ড তৈরি পানির মতো সহজ করে দিয়েছে।

CHAPTER 05

প্যাকেজ ইকোসিস্টেম, ডিপেন্ডেন্সি ও মডিউল সিস্টেম

প্যাকেজ ম্যানেজার (npm, pip), লক ফাইল, SemVer এবং ডিপেন্ডেন্সি হেল থেকে মুক্তির উপায়

৫.১ প্যাকেজ ম্যানেজার (Package Manager) কি?

সফটওয়্যার ইঞ্জিনিয়ারিংয়ের মূল নীতি হলো: "Never reinvent the wheel"। আপনি যখন পাসওয়ার্ড হ্যাশ করতে চান, পিডিএফ তৈরি করতে চান, কিংবা পেমেন্ট গেটওয়ের সাথে যুক্ত হতে চান—এগুলোর জন্য অন্য ইঞ্জিনিয়াররা ইতিমধ্যে নিখুঁত ও পরীক্ষিত লাইব্রেরি তৈরি করে রেখেছেন। এই লাইব্রেরিগুলোকে বলা হয় প্যাকেজ (Package) বা ডিপেন্ডেন্সি (Dependency)।

আর ইন্টারনেটের সেন্ট্রাল রেজিস্ট্রি থেকে এই লাইব্রেরিগুলো আপনার প্রজেক্টে স্বয়ংক্রিয়ভাবে ডাউনলোড, আপডেট ও পরিচালনা করার টুলকে বলা হয় প্যাকেজ ম্যানেজার।

> ⚡ EEE অ্যানালজি: সার্কিট ডিজাইনে BOM (Bill of Materials)
>
> আপনি যখন একটি PCB ডিজাইন করেন, তখন কম্পোনেন্ট লিস্ট তৈরি করেন যাকে বলে BOM (রেজিস্টর ১০০Ω ১০টি, ক্যাপাসিটর ১০μF ২টি, আইসি NE555 ১টি)। প্যাকেজ ম্যানেজার হলো সেই ডিজিটাল সাপ্লায়ার—আপনি লিস্ট লিখে দিবেন (`package.json` বা `requirements.txt`), সে সাথে সাথে সব কম্পোনেন্ট এনে আপনার প্রজেক্টের টেবিলে সাজিয়ে দেবে!

বিভিন্ন ল্যাঙ্গুয়েজের প্যাকেজ ম্যানেজারসমূহ

ল্যাঙ্গুয়েজ	টুলস	কনফিগ ফাইল	ইনস্টল কমান্ড
JavaScript / Node	`npm` , `pnpm` (সুপার ফাস্ট), `yarn`	`package.json`	`npm install` বা `pnpm add <pkg>`
Python	`pip` , `poetry` , `uv` (অত্যন্ত দ্রুত Rust-বেসড)	`requirements.txt` , `pyproject.toml`	`pip install -r requirements.txt`
Rust	`cargo`	`Cargo.toml`	`cargo build`

৫.২ `package.json` এবং এর ভেতরের উপাদান

একটি নোডজেএস বা রিঅ্যাক্ট প্রজেক্টের প্রাণকেন্দ্র হলো `package.json` ফাইল। এর দুটি গুরুত্বপূর্ণ অংশ থাকে:

- `dependencies` : অ্যাপটি প্রোডাকশনে রান হতে যেসব প্যাকেজ আবশ্যক (যেমন: `express` , `react` , `axios`)।
- `devDependencies` : শুধুমাত্র কোড লেখা ও টেস্টিংয়ের সময় দরকারি প্যাকেজ (যেমন: `typescript` , `eslint` , `vite`)। প্রোডাকশন সার্ভারে এগুলো ইনস্টল করার দরকার নেই।
- `scripts` : শর্টকাট কমান্ড। যেমন টার্মিনালে `npm run dev` লিখলে ভেতরে হয়তো `vite --host` রান হয়।

৫.৩ `node_modules` এবং লক ফাইলের রহস্য

আপনি যখন `npm install` চালান, তখন প্রজেক্টে একটি বিশালাকার ফোল্ডার তৈরি হয়— **`node_modules`** ।

> ⚠️ গোল্ডেন রুল: কখনই node_modules গিটহাবে পুশ করবেন না!
>
> `node_modules` ফোল্ডারের সাইজ সহজেই ৫০০MB থেকে ১GB হতে পারে। কারণ আপনি ১টি প্যাকেজ নামালে সে হয়তো পেছনের আরও ৫০টি সাব-প্যাকেজ নামায়! তাই এই ফোল্ডারটি সবসময় `.gitignore` ফাইলে দিয়ে রাখতে হয়। অন্য যে কেউ আপনার কোড নামিয়ে শুধু `npm install` দিলেই ইন্টারনেটের রেজিস্ট্রি থেকে সব ফাইল আবার তৈরি হয়ে যাবে।

লক ফাইল (`package-lock.json` বা `poetry.lock`) কেন জীবন বাঁচায়?

ধরা যাক, আপনার পিসিতে একটি প্যাকেজের ভার্সন ছিল `2.1.0`। ছয় মাস পর যখন ক্লাউড সার্ভারে কোডটি ডিপ্লয় করা হলো, তখন লাইব্রেরির নতুন ভার্সন এসেছে `2.2.0` যাতে কিছু ব্রেকিং চেঞ্জ (কোড কাজ না করা) আছে।

লক ফাইল প্রতিটি সাব-প্যাকেজের একেবারে নিখুঁত নির্দিষ্ট ভার্সন এবং তার ক্রিপ্টোগ্রাফিক হ্যাশ (Hash) আটকে রাখে। ফলে পৃথিবীর যে প্রান্তেই কোড রান করা হোক না কেন, হুবহু একই বাইট ইনস্টল হবে। তাই লক ফাইল অবশ্যই গিটহাবে পুশ করতে হয়।

৫.৪ সেমান্টিক ভার্সনিং (SemVer): 1.4.2 এর অর্থ কী?

সফটওয়্যার জগতে ভার্সন নম্বর তিনটি সংখ্যা দিয়ে লেখা হয়: MAJOR . MINOR . PATCH

- PATCH (যেমন 1.4.2 থেকে 1.4.3): কেবল ছোটখাটো বাগ ফিক্স করা হয়েছে। কোনো নতুন ফিচার আসেনি, আগের কোডে কোনো সমস্যা হবে না।
- MINOR (যেমন 1.4.2 থেকে 1.5.0): নতুন ফিচার যোগ হয়েছে, কিন্তু আগের সব কোড ঠিকমতো কাজ করবে (Backward Compatible)।
- MAJOR (যেমন 1.4.2 থেকে 2.0.0): বিশাল পরিবর্তন বা আর্কিটেকচার বদলে গেছে। পুরনো কোড ক্র্যাশ করতে পারে (Breaking Changes)।

`package.json` ফাইলে যদি চিহ্নের ব্যবহার দেখেন:

- `^1.4.2` (ক্যারট চিহ্ন): অটোমেটিক যে কোনো Minor ও Patch আপডেট নেবে (অর্থাৎ `1.9.9` পর্যন্ত নেবে, কিন্তু `2.0.0` এ যাবে না)।
- `~1.4.2` (টিল্ডে চিহ্ন): কেবল Patch আপডেট নেবে (অর্থাৎ `1.4.9` পর্যন্ত নেবে)।

৫.৫ ডিপেন্ডেন্সি হেল (Dependency Hell) এবং সমাধানের উপায়

কখনও কখনও আপনি এআই-এর কোড রান করার সময় দেখবেন:
`npm ERR! ERESOLVE unable to resolve dependency tree` বা `Peer dependency conflict`।

এর মানে হলো প্যাকেজ 'A' চায় প্যাকেজ 'C'-এর ভার্সন ১, কিন্তু প্যাকেজ 'B' চায় প্যাকেজ 'C'-এর ভার্সন ২। সিস্টেম সিদ্ধান্ত নিতে পারছে না কোনটিকে রাখবে।

> 🤖 ডিপেন্ডেন্সি কনফ্লিক্ট সমাধানের ফর্মুলা
>
> 1. সবচেয়ে দ্রুত সমাধান: টার্মিনালে `npm install --legacy-peer-deps` চালান। এটি অনাকাঙ্ক্ষিত কনফ্লিক্ট উপেক্ষা করে ইনস্টল করে।
> 2. প্যাকেজ ফ্রেশ ইনস্টল করতে:
>   লিনাক্স/ম্যাকে: `rm -rf node_modules package-lock.json && npm install`
>   উইন্ডোজে: `node_modules` ফোল্ডার ও লক ফাইল ডিলিট করে পুনরায় `npm install` দিন।

📌 অধ্যায় ৫ এর সারসংক্ষেপ

- প্যাকেজ ম্যানেজার ওপেন সোর্স লাইব্রেরি ম্যানেজ করে (npm, pip, cargo)।
- `package.json` হলো প্রজেক্টের ব্লুপ্রিন্ট ও BOM লিস্ট।
- `node_modules` কখনো গিটহাবে পুশ করবেন না, কিন্তু লক ফাইল (`package-lock.json`) অবশ্যই গিটহাবে রাখবেন।
- সেমান্টিক ভার্সনিং (MAJOR.MINOR.PATCH) দিয়ে লাইব্রেরির স্থিতিশীলতা বোঝা যায়।
- ডিপেন্ডেন্সি সমস্যা হলে ক্যাশ ডিলিট করে ফ্রেশ ইনস্টল বা `--legacy-peer-deps` ব্যবহার কার্যকর।

CHAPTER 06

ভার্সন কন্ট্রোল ও কোড কোলাবোরেশন: গিট (Git) ও গিটহাব (GitHub)

গিটের ৪টি স্তর, মূল কমান্ডসমূহ, ব্রাঞ্চিং, মার্জ কনফ্লিক্ট এবং গিটহাবের শক্তি

৬.১ গিট (Git) কেন দরকার? সফটওয়্যারের টাইম মেশিন

অতীতে মানুষ প্রজেক্টের কাজ করতে গিয়ে ফোল্ডার বানাত: `project_v1`, `project_v2_final`, `project_final_edited_final`। এটি এক চূড়ান্ত বিশৃঙ্খলা! কোনো নতুন কোড লিখে যদি পুরো সিস্টেম ক্র্যাশ করে, তবে আগের ঠিক অবস্থায় ফিরে যাওয়ার কোনো উপায় থাকত না।

Git হলো একটি ডিস্ট্রিবিউটেড ভার্সন কন্ট্রোল সিস্টেম (VCS)। এটি আপনার কোডের প্রতিটি পরিবর্তনের স্ন্যাপশট (Snapshot) বা ইতিহাস ক্রিপ্টোগ্রাফিক হ্যাশ দিয়ে নিখুঁতভাবে সংরক্ষণ করে। এটি সফটওয়্যারের টাইম মেশিনের মতো—আপনি চাইলে এক সেকেন্ডে ৩ মাস আগের যেকোনো ভার্সনে ফিরে যেতে পারেন।

> ⚡ EEE অ্যানালজি: স্কিম্যাটিক রিভিশন ও ফার্মওয়্যার ব্যাকআপ
>
> ইলেকট্রনিক্স প্রজেক্টে PCB রিভিশন (Rev 1.0, Rev 1.1, Rev 2.0) থাকে। যদি কোনো কম্পোনেন্ট পরিবর্তনের পর বোর্ডে শর্ট সার্কিট হয়, তবে ইঞ্জিনিয়াররা আগের রিভিশনে ফিরে যান। গিট আপনার কোডবেসের জন্য প্রতিটি লাইনের পরিবর্তনকে এমন মাইক্রো-রিভিশনে ধারণ করে রাখে!

৬.২ গিটের ৪টি রাজ্য (The Four Areas of Git)

গিটকে আয়ত্ত করতে হলে এর ভেতরের ৪টি স্তর স্পষ্টভাবে বুঝতে হবে:

1. Working Directory (ওয়ার্কিং ডিরেক্টরি): আপনার কম্পিউটারের সাধারণ ফোল্ডার যেখানে আপনি কোড লিখছেন বা এডিট করছেন (Unstaged changes)।
2. Staging Area / Index (স্টেজিং এরিয়া): যে যে ফাইলের পরিবর্তন আপনি পরবর্তী স্ন্যাপশটে অন্তর্ভুক্ত করতে চান, সেগুলোকে প্রস্তুত করে রাখা। কমান্ড: `git add .`
3. Local Repository (লোকাল রিপোজিটরি): আপনার কম্পিউটারের `.git` ডাটাবেজে স্থায়ী স্ন্যাপশট বা কমিট সংরক্ষিত হওয়া। কমান্ড: `git commit -m "মেসেজ"`
4. Remote Repository (রিমোট রিপোজিটরি): ইন্টারনেটের ক্লাউডে (যেমন GitHub, GitLab) কোডের ব্যাকআপ রাখা। কমান্ড: `git push`

৬.৩ অপরিহার্য গিট কমান্ডের চিট-শিট

কমান্ড	কাজ	বাস্তব প্রয়োগ
`git init`	বর্তমান ফোল্ডারে নতুন গিট রিপোজিটরি চালু করা	প্রজেক্টের শুরুতে একবার দিতে হয়
`git status`	কোন কোন ফাইল পরিবর্তিত হয়েছে তা দেখা	যেকোনো কমিটের আগে স্ট্যাটাস চেক করুন
`git add <file>` / `git add .`	ফাইলগুলোকে স্টেজিং এরিয়াতে তোলা	কমিটের জন্য প্রস্তুত করা
`git commit -m "বার্তা"`	একটি অর্থপূর্ণ স্ন্যাপশট স্থায়ীভাবে রেকর্ড করা	`git commit -m "Add user authentication"`
`git push origin <branch>`	লোকাল কোড ইন্টারনেটের গিটহাবে পাঠানো	টিমের সাথে কোড শেয়ার বা ব্যাকআপ রাখা
`git pull origin <branch>`	গিটহাব থেকে লেটেস্ট কোড লোকাল পিসিতে নামানো	কাজ শুরু করার আগে টিম মেম্বারদের কোড সিঙ্ক করা
`git branch <name>`	নতুন ব্রাঞ্চ তৈরি করা	মূল কোড সুরক্ষিত রেখে নতুন ফিচারে কাজ করা
`git checkout -b <branch>` / `git switch -c`	নতুন ব্রাঞ্চ তৈরি করে সেখানে চলে যাওয়া	আইসোলেটেড ফিচারে কাজ শুরু করা
`git stash`	অর্ধেক করা কাজ সাময়িকভাবে লুকিয়ে রাখা	জরুরি প্রয়োজনে অন্য ব্রাঞ্চে দ্রুত সুইচের জন্য

৬.৪ গিট বনাম গিটহাব: পার্থক্য কি?

অনেকে মনে করেন Git এবং GitHub একই জিনিস। কিন্তু তা নয়:

- Git: একটি ওপেন সোর্স সফটওয়্যার ও কমান্ড লাইন টুল যা আপনার লোকাল কম্পিউটারে রান করে ভার্সন নিয়ন্ত্রণ করে। ইন্টারনেট ছাড়াও গিট শতভাগ কাজ করে।
- GitHub: একটি ক্লাউড প্ল্যাটফর্ম (মাইক্রোসফটের মালিকানাধীন) যা আপনার গিট রিপোজিটরি ইন্টারনেটে হোস্ট করে, সুন্দর ওয়েব ইন্টারফেস দেয় এবং টিম কোলাবোরেশন ও অটোমেশন (Actions) সুবিধা প্রদান করে।

পুল রিকোয়েস্ট (Pull Request - PR) কি?

আপনি যখন কোনো ব্রাঞ্চে নতুন একটি ফিচার তৈরি শেষ করেন, আপনি সরাসরি মূল কোডে (main branch) যুক্ত করেন না। আপনি গিটহাবে একটি Pull Request খোলেন। এর অর্থ হলো: "আমি অমুক ফিচার বানিয়েছি, দয়া করে আমার কোড রিভিউ করুন এবং সব ঠিক থাকলে মূল কোডে মার্জ (Merge) করুন।"

৬.৫ মার্জ কনফ্লিক্ট (Merge Conflict) কেন হয় এবং সমাধান

মার্জ কনফ্লিক্ট তখনই হয় যখন দুজন মানুষ (অথবা আপনি এবং এআই) একই ফাইলের একই লাইনে ভিন্ন কোড লেখেন এবং গিট বুঝতে পারে না কোন লাইনটিকে সত্য বলে গ্রহণ করবে!

তখন ফাইলের ভেতরে এমন চিহ্ন দেখা যায়:

```
<<<<<<< HEAD
const apiUrl = "https://api.production.com";
=======
const apiUrl = "http://localhost:3000";
>>>>>>> feature-branch
```

সমাধান: ভয় পাওয়ার কিছু নেই। কোড এডিটরে গিয়ে যে লাইনটি রাখতে চান তা রেখে কনফ্লিক্ট মার্কারগুলো (`<<<<<<<`, `=======`, `>>>>>>>`) মুছে দিন। তারপর সেভ করে ফাইলটি আবার `git add .` এবং `git commit` করুন। ব্যস, কনফ্লিক্ট সমাধান!

৬.৬ `.gitignore` ফাইলের মহত্ত্ব

প্রজেক্টের রুটে `.gitignore` নামে একটি ফাইল থাকে। এতে যে যে ফাইলের নাম বা প্যাটার্ন লিখে দেওয়া হয়, গিট সেগুলোকে আজীবনের জন্য এড়িয়ে চলে (কখনই ট্র্যাকিং বা কমিট করে না)।

```
# অপরিহার্য .gitignore তালিকা
node_modules/
.env
dist/
build/
*.log
.DS_Store
```

📌 অধ্যায় ৬ এর সারসংক্ষেপ

- গিট হলো কোডের অফলাইন টাইম মেশিন; গিটহাব হলো কোড হোস্টিং ও টিমের মিলনমেলা।
- গিটের চক্র: Working Directory → (git add) → Staging → (git commit) → Local Repo → (git push) → GitHub।
- ফিচার বানানোর জন্য সবসময় নতুন ব্রাঞ্চ (git checkout -b feature-xyz) ব্যবহার করবেন।
- মার্জ কনফ্লিক্ট হলে ম্যানুয়ালি পছন্দের লাইনটি রেখে মার্কারগুলো কেটে দিয়ে নতুন কমিট করতে হয়।
- পাসওয়ার্ড ফাইল (.env) ও ভারী ফোল্ডার (node_modules) বাধ্যতামূলকভাবে .gitignore-এ রাখবেন।

CHAPTER 07

কন্টেইনারাইজেশন ও ডকার (Docker) সহজ ভাষায়

"আমার কম্পিউটারে চলে কিন্তু সার্ভারে চলে না" সংকটের অবসান: ডকারফাইল, ইমেজ, কন্টেইনার ও কম্পোজ

৭.১ ডকার কি এবং কেন এর জন্ম?

সফটওয়্যার ইন্ডাস্ট্রির সবচেয়ে প্রাচীন ও কুখ্যাত কৌতুক হলো:
— "ডেভেলপার: বস, অ্যাপটা সার্ভারে কেন ক্র্যাশ করল জানি না, আমার ল্যাপটপে তো দিব্যি চলছিল!"
— "বস: তাহলে তোমার ল্যাপটপটাই ক্লায়েন্টকে সার্ভার হিসেবে কুরিয়ার করে পাঠিয়ে দাও!"

কেন এই সমস্যা হয়? কারণ আপনার ল্যাপটপে হয়তো Python 3.11 ইনস্টল করা আছে, উইন্ডোজ ওএস চলছে, এবং নির্দিষ্ট কিছু C++ লাইব্রেরি ও পাথ কনফিগার করা আছে। কিন্তু প্রোডাকশন সার্ভারে হয়তো চলছে Ubuntu Linux যাতে পাইথনের পুরনো ভার্সন 3.8 দেওয়া এবং কিছু দরকারি প্যাকেজ নেই!

Docker হলো এমন একটি প্রযুক্তি যা আপনার পুরো অ্যাপ্লিকেশন, তার রানটাইম (যেমন Node.js বা Python), কোড, ডিপেন্ডেন্সি, লিনাক্স লাইব্রেরি এবং কনফিগারেশন—সবকিছুকে একটি স্বয়ংসম্পূর্ণ বাক্সে (Container) সিলগালা করে প্যাক করে দেয়। এই বাক্সটি আপনার ল্যাপটপ, আপনার বন্ধুর ম্যাকবুক কিংবা ক্লাউডের লিনাক্স সার্ভার—যেখানেই রান করবেন, হুবহু ১০০% একই আচরণ করবে।

> 💡 ঐতিহাসিক অ্যানালজি: মালবাহী জাহাজের শিপিং কন্টেইনার
>
> ১৯৫০ সালের আগে সমুদ্রপথে পণ্য পরিবহন ছিল এক দুঃস্বপ্ন। বস্তায় গম, কাঠের বাক্সে পিয়ানো আর ড্রামে তেল বোঝাই করা হতো। জাহাজ পরিবর্তন করার সময় এক ড্রাম গড়িয়ে গিয়ে পিয়ানো ভেঙে ফেলত!
>
> তারপর এলো এক যুগান্তকারী উদ্ভাবন: স্ট্যান্ডার্ড ইন্টারমোডাল শিপিং কন্টেইনার। কন্টেইনারের ভেতরে পিয়ানো থাকুক, চাল থাকুক বা গাড়ি থাকুক—বাইরে থেকে সব কন্টেইনারের সাইজ ও কর্নার লক হুবহু এক। যে ক্রেন জাহাজে কন্টেইনার তোলে, সেই ক্রেনই ট্রেনে বা ট্রাকে বসিয়ে দেয়।
>
> ডকার সফটওয়্যার জগতের সেই স্ট্যান্ডার্ড শিপিং কন্টেইনার!

> ⚡ EEE অ্যানালজি: স্বয়ংসম্পূর্ণ এমবেডেড মডিউল
>
> আপনি যদি একটি সার্কিটে কাঁচা ট্রানজিস্টর ও ক্যাপাসিটর দিয়ে রেডিও ফ্রিকোয়েন্সি রিসিভার বানান, তবে তাপমাত্রার তারতম্য বা ভোল্টেজ ফ্ল্যাকচুয়েশনে টিউনিং নষ্ট হতে পারে। কিন্তু আপনি যদি একটি প্রস্তুত ESP32 বা ব্লুটুথ মডিউল (যাতে নিজস্ব ক্রিস্টাল অসিলেটর, ভোল্টেজ রেগুলেটর ও অ্যান্টেনা সিল করা আছে) সরাসরি পিসিবির সকেটে প্লাগ করেন, তবে তা যে সার্কিটেই বসান না কেন শতভাগ নিখুঁতভাবে চলবে। ডকার কন্টেইনার হলো সফটওয়্যারের সেই সিলগালা শিল্ডেড মডিউল!

৭.২ ভার্চুয়াল মেশিন (VM) বনাম ডকার কন্টেইনার

অনেকে ভাবেন ডকার হয়তো VirtualBox বা VMware এর মতো ভার্চুয়াল মেশিন। কিন্তু তাদের আর্কিটেকচারে এক আকাশ-পাতাল মৌলিক পার্থক্য আছে:

বৈশিষ্ট্য	ভার্চুয়াল মেশিন (VM - VMware, VirtualBox)	ডকার কন্টেইনার (Docker Container)
আর্কিটেকচার	একটি হাইপারভাইজারের ওপর আস্ত একটি ভার্চুয়াল কম্পিউটার চলে, যার নিজস্ব পূর্ণাঙ্গ অপারেটিং সিস্টেম (Guest OS) থাকে।	কোনো আলাদা গেস্ট ওএস নেই! হোস্ট কম্পিউটারের লিনাক্স কার্নেল (Kernel) শেয়ার করে বিচ্ছিন্ন ইউজার স্পেসে চলে।
সাইজ	কয়েক গিগাবাইট (GB) থেকে শুরু হয় (উইন্ডোজ বা সম্পূর্ণ উবুন্টু ওএস ফাইল)।	খুব হালকা—মাত্র ৫০ মেগাবাইট (MB) থেকে ২০০ মেগাবাইটের মতো হতে পারে।
স্টার্টআপ গতি	বুট হতে ২ থেকে ৫ মিনিট সময় লাগে।	মিলিসেকেন্ডের মধ্যে (১-২ সেকেন্ডে) চালু হয়ে যায়।
রিসোর্স খরচ	RAM এবং CPU স্থায়ীভাবে দখল করে রাখে।	প্রয়োজন অনুযায়ী ডায়নামিকালি হোস্ট মেমোরি ব্যবহার করে।

৭.৩ ডকারের ৪টি মূল স্তম্ভ (Core Concepts)

ডকার বুঝতে হলে এই ৪টি পরিভাষা পানির মতো পরিষ্কার হতে হবে:

1. ১. Dockerfile (রেসিপি বা ব্লুপ্রিন্ট): এটি একটি সাধারণ টেক্সট ফাইল যাতে ধাপে ধাপে নির্দেশনা লেখা থাকে যে কন্টেইনারটি কীভাবে তৈরি হবে (কোন ওএস নেব, কোন কোড কপি করব, কী ইনস্টল করব)।
2. ২. Docker Image (প্রস্তুত প্যাকেট বা স্ন্যাপশট): ডকারফাইল বিল্ড করার পর যে রিড-অনলি বাইনারি প্যাকেজ তৈরি হয়। এটি সিডি-রম (CD-ROM) বা ইনস্টলেশন ফাইলের মতো স্থির।
3. ৩. Docker Container (জীবন্ত চলমান প্রসেস): ইমেজ থেকে যখন আপনি প্রোগ্রামটি রান করেন, তখন তৈরি হয় জীবন্ত কন্টেইনার। একই ইমেজ থেকে আপনি চাইলে ৫টি কন্টেইনার চালাতে পারেন।
4. ৪. Docker Hub / Registry (ইমেজ সংগ্রহশালা): ইন্টারনেটের অ্যাপ স্টোর, যেখান থেকে অফিশিয়াল পাইথন, নোডজেএস, পোস্টগ্রেস বা এনজিনিক্স ইমেজ এক ক্লিকে ডাউনলোড করা যায়।

একটি বাস্তবমুখী Dockerfile এর পোস্টমর্টেম

```
# ১. বেস ইমেজ নেওয়া (হালকা লিনাক্স যাতে নোডজেএস ২০ ইনস্টল আছে)
FROM node:20-alpine

# ২. কন্টেইনারের ভেতরের কাজের ডিরেক্টরি নির্ধারণ
WORKDIR /app

# ৩. প্যাকেজ কনফিগ ফাইল কপি করা
COPY package*.json ./

# ৪. ডিপেন্ডেন্সি ইনস্টল করা
RUN npm ci --only=production

# ৫. বাকি সব সোর্স কোড কপি করা
COPY . .

# ৬. পোর্ট উন্মুক্ত করার ঘোষণা (ডকুমেন্টেশন)
EXPOSE 3000

# ৭. কন্টেইনার চালু হওয়ার সময় মূল এক্সিকিউশন কমান্ড
CMD ["node", "server.js"]
```

৭.৪ `docker-compose.yml`: মাল্টি-কন্টেইনার অর্কেস্ট্রেশন

একটি বাস্তব পূর্ণাঙ্গ অ্যাপ্লিকেশনে কেবল একটি সার্ভিস থাকে না। সাধারণত থাকে:
১টি ফ্রন্টএন্ড (React), ১টি ব্যাকএন্ড এপিআই (Node/FastAPI), ১টি ডাটাবেজ (PostgreSQL) এবং ১টি ক্যাশ (Redis)।

এই ৪টি জিনিস আলাদা আলাদা রান না করে একটি ফাইলে লিখে এক কমান্ডে (`docker compose up`) চালু করার জাদুকরী টুলের নাম Docker Compose।

```
# docker-compose.yml ফাইলের উদাহরণ
version: '3.8'

services:
  # ব্যাকএন্ড সার্ভিস
  api:
    build: .
    ports:
      - "3000:3000"
    environment:
      - DATABASE_URL=postgresql://user:secret@db:5432/myapp
    depends_on:
      - db

  # ডাটাবেজ সার্ভিস (স্বয়ংক্রিয়ভাবে ক্লাউড থেকে অফিশিয়াল ইমেজ নামাবে)
  db:
    image: postgres:15-alpine
    environment:
      POSTGRES_USER: user
      POSTGRES_PASSWORD: secret
      POSTGRES_DB: myapp
    volumes:
      - pgdata:/var/lib/postgresql/data

volumes:
  pgdata: # কন্টেইনার ডিলিট হলেও ডাটাবেজের ডেটা হার্ডডিস্কে টিকে থাকবে!
```

৭.৫ ডকার ভলিউম (Volumes) ও পোর্ট ম্যাপিং (Port Mapping)

- পোর্ট ম্যাপিং (**`-p 8080:3000`**): কন্টেইনারের ভেতরের অ্যাপ হয়তো পোর্ট `3000` -এ চলছে। আপনার আসল ল্যাপটপের পোর্ট `8080` -এর সাথে এর সংযোগ তৈরি করাকে বলে পোর্ট ম্যাপিং। ফলে ব্রাউজারে `localhost:8080` লিখলে কন্টেইনারের অ্যাপে হিট হবে।
- ভলিউম (Volumes): কন্টেইনার ক্ষণস্থায়ী (Ephemeral)। কন্টেইনার স্টপ বা ডিলিট করলে তার ভেতরের ফাইল ধ্বংস হয়ে যায়। ডাটাবেজের ফাইল স্থায়ীভাবে হোস্ট মেশিনের হার্ডডিস্কে আটকে রাখতে ভলিউম ব্যবহার করা হয়।

৭.৬ দৈনন্দিন ডকার কমান্ডের চিট-শিট

কমান্ড	কাজ
`docker build -t myapp .`	বর্তমান ফোল্ডারের Dockerfile থেকে `myapp` নামে ইমেজ তৈরি করা
`docker run -d -p 3000:3000 --name web myapp`	ব্যাকগ্রাউন্ডে (Detached) পোর্ট ম্যাপিং করে কন্টেইনার চালানো
`docker ps`	বর্তমানে কোন কোন কন্টেইনার রানিং আছে তা দেখা
`docker logs -f <container_id>`	কন্টেইনারের ভেতরের লাইভ টার্মিনাল লগ বা এরর দেখা
`docker compose up -d`	সব সার্ভিস একসাথে ব্যাকগ্রাউন্ডে চালু করা
`docker compose down`	সব সার্ভিস ও তাদের নেটওয়ার্ক সুন্দরভাবে বন্ধ করে ফেলা

📌 অধ্যায় ৭ এর সারসংক্ষেপ

- ডকার সফটওয়্যারের সাথে তার যাবতীয় পরিবেশ সিল করে দেয়, ফলে যেকোনো কম্পিউটারে নির্ভুলভাবে চলে।
- ভার্চুয়াল মেশিনের মতো ডকার ভারী নয়; এটি হোস্ট ওএস কার্নেল শেয়ার করে বিদ্যুৎগতিতে চলে।
- Dockerfile হলো রেসিপি, Image হলো হিমায়িত স্ন্যাপশট, আর Container হলো লাইভ এক্সিকিউশন।
- `docker-compose` দিয়ে ব্যাকএন্ড, ডাটাবেজ ও ক্যাশ এক ক্লিকে একসাথে নিয়ন্ত্রণ করা যায়।
- ভলিউম ব্যবহার করলে কন্টেইনার ধ্বংস হলেও ডাটাবেজের তথ্য হার্ডডিস্কে সুরক্ষিত থাকে।

CHAPTER 08

অটোমেশন ও সিআই/সিডি: গিটহাব অ্যাকশনস (GitHub Actions)

ম্যানুয়াল ডেপ্লয়মেন্টের দিন শেষ: ওয়ার্কফ্লো, রানার, সিক্রেটস এবং অটোমেটেড টেস্টিং পাইপলাইন

৮.১ সিআই/সিডি (CI/CD) কি? সফটওয়্যারের আধুনিক অ্যাসেম্বলি লাইন

আগেকার দিনে সফটওয়্যার রিলিজ করার প্রক্রিয়া ছিল চরম ক্লান্তিকর ও ঝুঁকিপূর্ণ:
১. ডেভেলপার নিজের ল্যাপটপে কোড লিখতেন।
২. ম্যানুয়ালি টেস্ট চালাতেন।
৩. এফটিপি (FTP) বা এসএসএইচ (SSH) দিয়ে লাইভ প্রোডাকশন সার্ভারে লগইন করতেন।
৪. মাউস টেনে টেনে ফাইল কপি করতেন বা ম্যানুয়ালি কমান্ড চালাতেন।

মাঝরাতে সার্ভারে একটি ভুল ফাইলের কারণে পুরো ওয়েবসাইট ডাউন হয়ে যেত! এই বিশৃঙ্খলা দূর করার আধুনিক সমাধান হলো CI/CD:

- Continuous Integration (CI): আপনি যখনই গিটহাবে নতুন কোড পুশ করবেন, স্বয়ংক্রিয়ভাবে একটি রোবট সিস্টেম কোডটি নামিয়ে কম্পাইল করবে, বিল্ড করবে এবং শত শত অটোমেটেড টেস্ট রান করে যাচাই করবে কোনো বাগ আছে কিনা।
- Continuous Deployment (CD): যদি সব টেস্ট সফলভাবে পাস করে (সবুজ টিকমার্ক ✅), তবে কোনো মানুষের স্পর্শ ছাড়াই স্বয়ংক্রিয়ভাবে কোডটি লাইভ ক্লাউড সার্ভারে চলে যাবে এবং ইউজাররা নতুন ফিচার পেয়ে যাবেন!

> ⚡ EEE অ্যানালজি: স্বয়ংক্রিয় PCB উৎপাদন ও টেস্টিং লাইন
>
> একটি আধুনিক ইলেকট্রনিক্স কারখানায় সার্কিট বোর্ড কীভাবে তৈরি হয় ভাবুন:
> ১. রোবোটিক Pick-and-Place মেশিন নিখুঁতভাবে কম্পোনেন্ট বসায়।
> ২. রিফ্লো ওভেনে শোল্ডারিং হয়।
> ৩. Automated Optical Inspection (AOI) ক্যামেরা বোর্ড স্ক্যান করে দেখে কোনো শর্ট সার্কিট বা ভুল পিন আছে কিনা (এটি হলো CI)।
> ৪. যদি বোর্ড শতভাগ পাস করে, কনভেয়ার বেল্ট স্বয়ংক্রিয়ভাবে এটিকে প্যাকেজিং ও শিপিং ইউনিটে পাঠিয়ে দেয় (এটি হলো CD)।
>
> GitHub Actions হলো সফটওয়্যারের সেই রোবোটিক কনভেয়ার বেল্ট ও ইনস্পেকশন লাইন!

৮.২ গিটহাব অ্যাকশনস (GitHub Actions) কীভাবে কাজ করে?

GitHub Actions হলো গিটহাবের নিজস্ব বিল্ট-ইন অটোমেশন ও সিআই/সিডি প্ল্যাটফর্ম। আপনি রিপোজিটরির `.github/workflows/` ফোল্ডারের ভেতরে একটি `YAML (.yml)` ফাইল রেখে দিলেই গিটহাব ক্লাউড ইঞ্জিন স্বয়ংক্রিয়ভাবে কাজ শুরু করে দেয়।

অ্যাকশনসের ৬টি মূল উপাদান (Core Anatomy)

1. Workflows (ওয়ার্কফ্লো): আপনার অটোমেশন প্রক্রিয়ার সম্পূর্ণ ব্লুপ্রিন্ট। যেমন: "টেস্টিং ওয়ার্কফ্লো" বা "প্রোডাকশন ডেপ্লয় ওয়ার্কফ্লো"।
2. Events / Triggers (ইভেন্ট): কোন ঘটনায় ওয়ার্কফ্লো চালু হবে। যেমন: `on: [push, pull_request]`। অর্থাৎ কেউ নতুন কোড পুশ করলেই এটি জেগে উঠবে।
3. Jobs (জব): ওয়ার্কফ্লোর ভেতরে কাজের সেট। ডিফল্টভাবে একাধিক জব সমান্তরালে (Parallel) চলে। যেমন: এক জবে ফ্রন্টএন্ড টেস্ট, অন্য জবে ব্যাকএন্ড টেস্ট।

4. Runners (রানার): গিটহাবের ক্লাউডে চালু হওয়া একটি সম্পূর্ণ ভার্চুয়াল কম্পিউটার (যেমন: `runs-on: ubuntu-latest`)। গিটহাব ফ্রিতে আপনাকে এই উবুন্টু মেশিন দেয় আপনার কোড রান করার জন্য! কাজ শেষ হলে মেশিনটি নিজে থেকেই ধ্বংস হয়ে যায়।
5. Steps (স্টেপস): একটি জবের ভেতরের ক্রমান্বয়ে ঘটা কাজ (যেমন: ডিপেন্ডেন্সি ইনস্টল করা, টেস্ট চালানো)।
6. Actions (অ্যাকশন): তৈরি করা প্লাগইন বা মডিউল। যেমন: গিট রিপোজিটরি ক্লোন করার জন্য কমিউনিটি প্লাগইন `actions/checkout@v4`।

৮.৩ একটি বাস্তব প্রোডাকশন ওয়ার্কফ্লো ফাইলের লাইন-বাই-লাইন ব্যবচ্ছেদ

নিচের ফাইলটি লক্ষ্য করুন (পাথ: `.github/workflows/ci-deploy.yml`):

```
name: Automated Build and Test

# ১. কোন ইভেন্টে এই পাইপলাইন চলবে?
on:
  push:
    branches: [ "main" ]
  pull_request:
    branches: [ "main" ]

# ২. কাজের তালিকা
jobs:
  test_and_verify:
    runs-on: ubuntu-latest # ৩. ক্লাউড লিনাক্স কম্পিউটার

    steps:
      # স্টেপ ১: গিটহাবের কোডটি রানার কম্পিউটারে নামিয়ে নাও
      - name: Check out repository code
        uses: actions/checkout@v4

      # স্টেপ ২: রানার কম্পিউটারে নোডজেএস ২০ প্রস্তুত করো
      - name: Setup Node.js Environment
        uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: 'npm'

      # স্টেপ ৩: ডিপেন্ডেন্সি ইনস্টল করো
      - name: Install Dependencies
        run: npm ci

      # স্টেপ ৪: টাইপস্ক্রিপ্ট টাইপ চেক ও অটোমেটেড টেস্ট চালাও
      - name: Run Automated Tests
        run: npm test

      # স্টেপ ৫: প্রোডাকশন বিল্ড বানাও
      - name: Build Production Bundle
        run: npm run build
```

৮.৪ গিটহাব সিক্রেটস (GitHub Secrets): ক্লাউড পাসওয়ার্ড ম্যানেজমেন্ট

সিআই/সিডি দিয়ে যখন অটোমেটিক এডব্লিউএস (AWS) সার্ভারে ফাইল আপলোড করবেন বা ডকার হাবে ইমেজ পুশ করবেন, তখন পাসওয়ার্ড বা এপিআই কি প্রয়োজন হবে।

⚠️ সাবধান: YAML ফাইলে কখনই পাসওয়ার্ড টাইপ করবেন না!

কারণ গিট রিপোজিটরি যে কেউ দেখলে পাসওয়ার্ড চুরি হয়ে যাবে। এর সমাধান হলো:
গিটহাব রিপোজিটরির Settings → Secrets and variables → Actions -এ যান। সেখানে `DOCKER_PASSWORD` বা `SERVER_SSH_KEY` নামে ভ্যারিয়েবল যোগ করুন।
YAML ফাইলে তা ব্যবহার করুন এভাবে:
`password: ${{ secrets.DOCKER_PASSWORD }}`
গিটহাব ক্লাউড টার্মিনালে এই মানটি তারকাচিহ্ন (***) হিসেবে প্রিন্ট করবে, কেউ কখনই দেখতে পাবে না।

৮.৫ যখন গিটহাব অ্যাকশনসে লাল ক্রস (❌) আসে, তখন কী করবেন?

কোড পুশ করার পর আপনার কমিটের পাশে হলুদ গোল চিহ্ন ঘোরে (কাজ চলছে)। যদি সবুজ টিকমার্ক আসে (✅), তার মানে সবকিছু পারফেক্ট। কিন্তু যদি লাল ক্রস আসে (❌):

1. গিটহাব রিপোজিটরির Actions ট্যাবে ক্লিক করুন।
2. লাল ক্রস হওয়া স্পেসিফিক রানটির ওপর ক্লিক করুন।
3. যে স্টেপের পাশে লাল দাগ আছে সেটিতে ক্লিক করলে টার্মিনাল লগ দেখতে পাবেন।
4. লগের শেষ কয়েকটি লাইনে লাল রঙে স্পষ্টভাবে লেখা থাকে এররটি কী (যেমন: কোনো টেস্ট ফেইল করেছে নাকি প্যাকেজ খুঁজে পায়নি)।

🤖 এআই দিয়ে গিটহাব অ্যাকশন ফিক্স করার প্রম্পট

টার্মিনাল থেকে লাল এরর লগটি কপি করে এআই-কে বলুন:
"My GitHub Actions workflow failed at step 'Run Automated Tests' with the following error log: [লগ পেস্ট করুন]. Here is my .github/workflows/deploy.yml file. Please diagnose why it failed and provide the exact fix."

📌 অধ্যায় ৮ এর সারসংক্ষেপ

- সিআই/সিডি হলো সফটওয়্যারের রোবোটিক কোয়ালিটি কন্ট্রোল ও অটোমেটেড শিপিং লাইন।
- GitHub Actions স্বয়ংক্রিয়ভাবে `.github/workflows/*.yml` ফাইল দেখে পরিচালিত হয়।
- গিটহাব ক্লাউডে ফ্রি লিনাক্স কম্পিউটার (Runner) দেয় কোড বিল্ড ও টেস্ট করার জন্য।
- পাসওয়ার্ড ও এপিআই কি রাখতে হয় GitHub Repository Secrets-এ, কোডে নয়।
- লাল ক্রস দেখলে অ্যাকশন লগের শেষ লাইন পড়ে সহজেই এআই দিয়ে সমস্যা সমাধান করা যায়।

CHAPTER 09

ক্লাউড কম্পিউটিং, সার্ভার কনফিগারেশন ও লাইভ ডেপ্লয়মেন্ট

VPS বনাম PaaS, রিভার্স প্রক্সি Nginx, ডোমেন ও SSL, এবং প্রসেস ম্যানেজার PM2

৯.১ হোস্টিং জগতের মানচিত্র: আপনার কোড কোথায় থাকবে?

আপনার ল্যাপটপে লোকালহোস্টে (`localhost:3000`) চলা অ্যাপটি যখন সারা বিশ্বের কোটি কোটি মানুষের ব্যবহারের জন্য উন্মুক্ত করবেন, তখন ইন্টারনেটের সাথে সার্বক্ষণিক উচ্চগতির অপটিক্যাল ফাইবার যুক্ত কোনো কম্পিউটারে সেটিকে চালু রাখতে হবে। একে বলা হয় ডেপ্লয়মেন্ট (Deployment)।

প্ল্যাটফর্ম ক্যাটাগরি	উদাহরণ	কীভাবে কাজ করে?	কাদের জন্য সেরা?
PaaS (Platform as a Service)	Vercel, Render, Railway	জিরো-কনফিগারেশন। শুধু গিটহাব রিপোজিটরি লিংক করবেন, কোড পুশ করলেই স্বয়ংক্রিয়ভাবে লাইভ ইউআরএল তৈরি হয়ে যায়।	এআই বিল্ডার, দ্রুত প্রোটোটাইপিং ও ফ্রন্টএন্ড/ফুলস্ট্যাকের জন্য পরম আশীর্বাদ।
VPS (Virtual Private Server)	DigitalOcean Droplet, Hetzner, AWS EC2	আপনাকে একটি রুট পাসওয়ার্ডসহ ফাঁকা লিনাক্স কম্পিউটার দেওয়া হবে। Nginx, ডকার, ডাটাবেজ—সব নিজেকে সেটআপ করতে হবে।	বাজেট ফ্রেন্ডলি (মাত্র $৪-৫/মাস), পূর্ণ স্বাধীনতা ও দীর্ঘমেয়াদী প্রোডাকশন প্রজেক্ট।
Serverless / Edge	Cloudflare Workers, AWS Lambda	কোনো সার্বক্ষণিক সার্ভার চলে না। কেবল যখন কোনো ইউজার রিকোয়েস্ট করে, তখন কয়েক মিলিসেকেন্ডের জন্য কোড এক্সিকিউট হয়ে বন্ধ হয়ে যায়।	কম খরচে কোটি কোটি রিকোয়েস্ট হ্যান্ডল করার জন্য।

> 🤖 এআই বিল্ডারদের জন্য সহজ ডেপ্লয়মেন্ট সুপারিশ
>
> ফ্রন্টএন্ড বা Next.js অ্যাপের জন্য সরাসরি Vercel ব্যবহার করুন (ফ্রি টায়ার চমৎকার)। আর ব্যাকএন্ড এপিআই বা ডকার প্রজেক্টের জন্য Render বা Railway বেছে নিন। মাত্র ৫ মিনিটে লাইভ লিঙ্ক পাওয়া যায়!

৯.২ রিভার্স প্রক্সি (Reverse Proxy) ও Nginx কেন লাগে?

ধরা যাক, আপনি একটি VPS সার্ভারে আপনার নোডজেএস অ্যাপ রান করেছেন যা পোর্ট `3000`-এ চলছে। কিন্তু কোনো সাধারণ ব্যবহারকারী কি ব্রাউজারে `http://159.65.12.84:3000` টাইপ করবেন? কখনোই না! তারা ব্রাউজারে লিখবেন: `https://myawesomeapp.com`।

ইন্টারনেটের স্ট্যান্ডার্ড পোর্ট হলো `80` (HTTP) এবং `443` (HTTPS)। কিন্তু আপনার নোড অ্যাপ চলছে `3000`-এ।

> 💡 কর্পোরেট অফিসের রিসেপশনিস্ট মডেল
>
> বাইরে থেকে কোনো ভিজিটর এলে সরাসরি কোনো ম্যানেজারের রুমে ঢুকতে পারেন না। রিসেপশনে (পোর্ট ৮০/৪৪৩) একজন স্মার্ট রিসেপশনিস্ট বসেন। তিনি ভিজিটরের নাম শুনে সঠিক রুমে (পোর্ট ৩০০০) পাঠিয়ে দেন।
>
> Nginx (ইঞ্জিন-এক্স) হলো ইন্টারনেটের সেই সেরা রিসেপশনিস্ট বা Reverse Proxy। এটি বাইরের ট্রাফিক গ্রহণ করে ভেতরের নির্দিষ্ট পোর্টে রাউট করে দেয়।

> ⚡ EEE অ্যানালজি: স্টেপ-ডাউন সাবস্টেশন ও ট্রান্সফরমার
>
> ন্যাশনাল গ্রিড থেকে ১১,০০০ ভোল্টের হাই-টেনশন বিদ্যুৎ সরাসরি ঘরের ফ্যানে দিলে পুড়ে যাবে। সাবস্টেশন ও ট্রান্সফরমার যেমন বাইরের হাই-ভোল্টেজ সিগন্যাল গ্রহণ করে গৃহস্থালির জন্য উপযোগী ২২০ ভোল্টে রূপান্তরিত করে নিরাপদ সার্কিটে পাঠায়, Nginx তেমনি ইন্টারনেটের পোর্ট ৪৪৩ এর ট্রাফিক ও SSL এনক্রিপশন সামলে ভেতরের লোকাল অ্যাপ্লিকেশনে মসৃণভাবে বিতরণ করে!

একটি সাধারণ Nginx কনফিগারেশন ব্লকের নমুনা

```
server {
    listen 80;
    server_name myawesomeapp.com;

    location / {
        proxy_pass http://localhost:3000; # ট্রাফিক পাঠিয়ে দাও ৩০০০ পোর্টে
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
        proxy_set_header Host $host;
        proxy_cache_bypass $http_upgrade;
    }
}
```

৯.৩ ডোমেন, ডিএনএস রেকর্ডস ও ফ্রি SSL (Certbot)

যখন আপনি Namecheap বা GoDaddy থেকে একটি ডোমেন কেনেন (যেমন `example.com`), তখন ডোমেনের ডিএনএস সেটিংসে গিয়ে সার্ভারের আইপি যোগ করতে হয়:

- A Record: ডোমেন নামকে সার্ভারের IPv4 আইপিতে নির্দেশ করে (যেমন: `@ → 159.65.12.84`)।
- CNAME Record: একটি সাবডোমেনকে অন্য ডোমেনে পয়েন্ট করে (যেমন: `www → example.com`)।

ব্রাউজারে সবুজ তালাচাবি (HTTPS / SSL):

আগে SSL সার্টিফিকেটের জন্য বছরে হাজার হাজার টাকা খরচ হতো। এখন Let's Encrypt এবং Certbot টুলের মাধ্যমে লিনাক্স টার্মিনালে মাত্র এক লাইনের কমান্ড দিয়ে আজীবনের জন্য ফ্রি অটো-রিনিউয়িং SSL নেওয়া যায়:

`sudo certbot --nginx -d myawesomeapp.com`

৯.৪ প্রসেস ম্যানেজার: PM2

টার্মিনালে যদি আপনি `node server.js` লিখে SSH টার্মিনাল উইন্ডো কেটে দেন, নোড অ্যাপটি সাথে সাথে বন্ধ হয়ে যাবে। আবার কোনো অচিন্ত্যনীয় বাগের কারণে কোডে এরর হলে অ্যাপ ক্র্যাশ করে স্থায়ীভাবে বন্ধ হয়ে থাকবে।

PM2 হলো প্রোডাকশন প্রসেস ম্যানেজার। এটি আপনার অ্যাপকে ব্যাকগ্রাউন্ডে অমর রাখে—সার্ভার রিস্টার্ট হলেও অ্যাপ স্বয়ংক্রিয়ভাবে চালু হয় এবং কোড কোনো কারণে ক্র্যাশ করলে মিলিসেকেন্ডের মধ্যে পুনরায় রিস্টার্ট দেয়।

```
# PM2 কমান্ডসমূহ
npm install -g pm2
pm2 start server.js --name "my-backend"
pm2 save
pm2 startup # সার্ভার রিবুট হলেও স্বয়ংক্রিয়ভাবে চালু করার হুক
```

📌 অধ্যায় ৯ এর সারসংক্ষেপ

- Vercel বা Render দিয়ে এআই প্রজেক্ট এক ক্লিকে গিটহাব থেকে সরাসরি লাইভ ডেপ্লয় করা যায়।
- VPS সার্ভারে Nginx রিভার্স প্রক্সি হিসেবে পোর্ট ৮০/৪৪৩ থেকে লোকাল পোর্টে ট্রাফিক পরিচালনা করে।
- ডিএনএস এর 'A Record' ডোমেনকে সার্ভারের আইপির সাথে সংযুক্ত করে।
- Certbot দিয়ে সম্পূর্ণ ফ্রিতে এনক্রিপ্টেড HTTPS (SSL সার্টিফিকেট) যুক্ত করা যায়।
- PM2 প্রসেস ম্যানেজার সার্ভারে অ্যাপকে ক্র্যাশ ও রিবুট থেকে বাঁচিয়ে সার্বক্ষণিক সচল রাখে।

CHAPTER 10

সিস্টেম ডিজাইন ও সফটওয়্যার আর্কিটেকচার প্যাটার্নস

মোনোলিথ বনাম মাইক্রোসার্ভিস, মেসেজ কিউ, ক্যাশিং, স্কেলিং এবং হাই-অ্যাভেইলেবিলিটি

১০.১ মোনোলিথ (Monolith) বনাম মাইক্রোসার্ভিস (Microservices)

সফটওয়্যার জগতে সবচেয়ে বড় তর্কের বিষয় এটি। দুটি আর্কিটেকচারের সুবিধা ও সীমাবদ্ধতা বোঝা যাক:

- মোনোলিথিক আর্কিটেকচার (Monolith): সম্পূর্ণ সিস্টেম—অথেন্টিকেশন, পেমেন্ট, প্রোডাক্ট ক্যাটালগ, নোটিফিকেশন—সবকিছু একটিমাত্র বিশাল কোডবেসে এবং একটিমাত্র সার্ভারে রান করে।
  সুবিধা: ডেভেলপ করা খুব সহজ, ডেপ্লয়মেন্ট সহজ, নেটওয়ার্ক ল্যাটেন্সি নেই।
  অসুবিধা: সিস্টেম বিশাল হলে একটি ছোট ভুলে পুরো অ্যাপ ক্র্যাশ করতে পারে।
- মাইক্রোসার্ভিসেস আর্কিটেকচার (Microservices): পুরো অ্যাপকে ছোট ছোট স্বাধীন মিনি-সার্ভিসে ভাগ করা হয় (যেমন: Auth Service, Billing Service, Order Service)। প্রতিটি সার্ভিসের নিজস্ব ডাটাবেজ থাকতে পারে এবং তারা নেটওয়ার্কে REST/gRPC দিয়ে কথা বলে।
  সুবিধা: Netflix বা Uber-এর মতো শত শত ইঞ্জিনিয়ারের টিমের জন্য সেরা। একটি সার্ভিস ডাউন হলেও অন্যগুলো চলতে পারে।
  অসুবিধা: অতিরিক্ত জটিলতা, নেটওয়ার্ক এরর ও ডেভঅপ্স মেইনটেনেন্স খরচ অনেক বেশি।

> ⚡ EEE অ্যানালজি: সেন্ট্রাল পাওয়ার গ্রিড বনাম ডিস্ট্রিবিউটেড সোলার মাইক্রোগ্রিড
>
> মোনোলিথ হলো একটি একক বিশালাকার কয়লাভিত্তিক বিদ্যুৎকেন্দ্র—এক জায়গায় পাওয়ার বন্ধ হলে গোটা দেশ অন্ধকার! আর মাইক্রোসার্ভিস হলো ছোট ছোট সোলার রুফটপ ও ব্যাটারি স্টোরেজ মাইক্রোগ্রিডের মতো—একটি ছাদের প্যানেল নষ্ট হলেও পাড়ার বাকি বাড়িগুলোতে আলো জ্বলতেই থাকে। তবে সব মাইক্রোগ্রিডকে সিঙ্ক করা বেশ জটিল।

> ⚠️ নতুন প্রজেক্টের জন্য এআই-এর বিপজ্জনক ফাঁদ
>
> এআই চ্যাটে প্রায়ই নতুন প্রজেক্টের জন্যও হাইপ দেখে বলবে: "আসুন আমরা ডজনখানেক মাইক্রোসার্ভিস দিয়ে শুরু করি!"। কখনও এই ফাঁদে পা দেবেন না! শুরুতে সবসময় একটি পরিষ্কার মোনোলিথ (Modular Monolith) দিয়ে শুরু করবেন। যখন আপনার লক্ষাধিক নিয়মিত অ্যাক্টিভ ইউজার হবে, তখন প্রয়োজন অনুসারে ধীরে ধীরে ভারী অংশগুলোকে আলাদা সার্ভিসে রূপান্তর করবেন।

১০.২ ব্যাকগ্রাউন্ড জব ও মেসেজ কিউ (Message Queues)

ধরা যাক, একজন ইউজার আপনার সাইটে সাইনআপ করলেন। আপনার অ্যাপকে করতে হবে:
১. ডাটাবেজে ইউজার সেভ করা (১০ মিলিসেকেন্ড)।
২. একটি সুন্দর ওয়েলকাম পিডিএফ জেনারেট করা (৩ সেকেন্ড)।
৩. জিমেইলে SMTP দিয়ে ইমেইল পাঠানো (২ সেকেন্ড)।

যদি এই ৫ সেকেন্ড ইউজারকে ব্রাউজারে গোল গোল চাকা ঘুরিয়ে বসিয়ে রাখেন, ইউজার বিরক্ত হয়ে সাইট বন্ধ করে দেবেন!

সমাধান: Message Queue (মেসেজ কিউ)। রিকোয়েস্ট আসা মাত্র ইউজারকে ২ মিলিসেকেন্ডে `200 OK - Registration Successful` মেসেজ দিয়ে দিন। আর পেছনের ভারী কাজগুলো (পিডিএফ বানানো, ইমেইল পাঠানো) একটি কিউ বা লাইনে (Queue) জমা করে দিন। ব্যাকগ্রাউন্ডে অলস বসে থাকা অন্য একটি প্রসেস (Worker) লাইন থেকে একটি একটি করে কাজ নিয়ে শান্তিতে সম্পন্ন করবে।

জনপ্রিয় কিউ ও ব্রোকার টুলস: Redis Queue (BullMQ) (Node.js), Celery (Python), RabbitMQ, এবং Apache Kafka (বিলিয়ন ইভেন্ট স্ট্রিম)।

১০.৩ ক্যাশিং আর্কিটেকচার: Redis এর জাদু

ডাটাবেজ ডিস্কে তথ্য রাখে, যা র‍্যামের তুলনায় অনেক ধীরগতির। একই প্রোডাক্ট পেজে যদি প্রতি সেকেন্ডে ১,০০০ ইউজার ঢোকেন, ডাটাবেজ এক সেকেন্ডে ১,০০০ বার একই এসকিউএল কুয়েরি চালাতে গিয়ে থমকে যাবে।

> 💡 লাইব্রেরি বনাম পড়ার টেবিল
>
> প্রতিটি বই পড়ার জন্য বারবার দোতলার লাইব্রেরির সেলফে যাওয়া হলো ডাটাবেজ কুয়েরি। আর চলতি সপ্তাহে যে ৩টি বই বারবার দরকার হয় সেগুলো নিজের পড়ার টেবিলে সামনে সাজিয়ে রাখাই হলো ক্যাশ (Cache)!

Redis মেমোরিতে (RAM) ডেটা কি-ভ্যালু আকারে রাখে। ফলে কুয়েরির উত্তর পেতে ১ মিলিসেকেন্ডেরও কম সময় লাগে। প্রথম রিকোয়েস্টে ডাটাবেজ থেকে ডেটা এনে Redis-এ লিখে রাখা হয় (TTL বা টাইম-টু-লাইভসহ, যেমন ১০ মিনিট)। পরবর্তী ৯৯৯টি রিকোয়েস্ট সরাসরি রেডিস থেকে সার্ভ হয়ে যায়—ডাটাবেজ পুরোপুরি মুক্ত থাকে!

১০.৪ স্কেলিং: Vertical বনাম Horizontal

ফিচার	ভার্টিক্যাল স্কেলিং (Scale Up)	হরাইজন্টাল স্কেলিং (Scale Out)
পদ্ধতি	বিদ্যমান একটি সার্ভারেই আরও বেশি RAM (৮GB থেকে ৬৪GB) ও শক্তিশালী CPU যোগ করা।	একটি সার্ভারের বদলে পাশাপাশি একইরকম ৪-৫টি মাঝারি মানের সার্ভার বসানো।
সীমাবদ্ধতা	হার্ডওয়্যারের একটি সর্বোচ্চ ক্ষমতা আছে এবং অত্যন্ত ব্যয়বহুল। তাছাড়া সার্ভার বন্ধ হলে সাইট ডাউন (Single Point of Failure)।	তাত্ত্বিকভাবে অসীম স্কেল করা যায়। একটি সার্ভার নষ্ট হলে বাকিগুলো চালু থাকে (High Availability)।
প্রয়োজনীয় টুল	কোনো স্পেশাল টুলের প্রয়োজন নেই।	সামনে একটি Load Balancer (লোড ব্যালেন্সার) লাগে যা রাউন্ড-রবিন নিয়মে ট্রাফিক ভাগ করে দেয়।

> 📌 অধ্যায় ১০ এর সারসংক্ষেপ
>
> - শুরুতে সবসময় ক্লিন মোনোলিথ ব্যবহার করুন; অহেতুক মাইক্রোসার্ভিস দিয়ে জটিলতা বাড়াবেন না।
> - ভারী কাজ (ইমেইল পাঠানো, পিডিএফ জেনারেট) সরাসরি মেইন থ্রেডে না রেখে মেসেজ কিউ (BullMQ/Celery) দিয়ে ব্যাকগ্রাউন্ডে দিন।
> - Redis ক্যাশিং ব্যবহার করে ডাটাবেজের লোড ৯০% কমিয়ে সাইটের গতি ১০ গুণ বাড়ানো যায়।
> - ভার্টিক্যাল স্কেলিং মানে সার্ভারের পাওয়ার বাড়ানো, হরাইজন্টাল স্কেলিং মানে লোড ব্যালেন্সারের পেছনে একাধিক সার্ভার বসানো।

CHAPTER 11

এআই দিয়ে সফটওয়্যার তৈরির প্র্যাকটিক্যাল মাস্টারক্লাস ও ডিব্যাগিং মাইন্ডসেট

কনটেক্সট ইঞ্জিনিয়ারিং, আর্কিটেক্ট প্রম্পটিং, স্ট্যাক ট্রেস ডিব্যাগিং এবং প্রোডাকশন রোডম্যাপ

১১.১ এআই কোডিং টুলের দুনিয়া: আপনি চালক, এআই ইঞ্জিন

আজকের দিনে বাজারে শীর্ষস্থানীয় এআই কোডিং টুলস যেমন Cursor, Claude Sonnet 3.5/3.7, Windsurf, ChatGPT, এবং Antigravity কোড লেখার গতিকে ১০ গুণ বাড়িয়ে দিয়েছে। কিন্তু বহু নন-সিএস মানুষ এগুলো নিয়ে কাজ করতে গিয়ে আধাঘণ্টার মধ্যে হতাশ হয়ে পড়েন। কেন?

কারণ তারা এআই-কে বলেন: "আমাকে একটি ইউটিউবের মতো সাইট বানিয়ে দাও!"

এআই কোনো জাদুকর নয়; এটি একটি প্যাটার্ন ম্যাচিং ও লজিক্যাল মডেল। আপনি যত অস্পষ্ট নির্দেশ দেবেন, এআই তত মনগড়া অনুমান (Hallucination) করবে। আপনাকে একজন দক্ষ প্রজেক্ট ম্যানেজার বা সফটওয়্যার আর্কিটেক্টের মতো ক্ষুদ্রাতিক্ষুদ্র ধাপে ধাপে এআই পরিচালনা করতে হবে।

১১.২ কনটেক্সট ইঞ্জিনিয়ারিং: এআই-কে সঠিক ব্রিফিং দেওয়া

একটি নিখুঁত এআই আর্কিটেকচারাল প্রম্পটের কাঠামো হওয়া উচিত ৫টি উপাদানের সমন্বয়ে:

1. ভূমিকা (Role): "You are a Senior Fullstack Architect and DevOps Specialist."
2. প্রজেক্টের প্রেক্ষাপট (Context): "I am building an automated invoice generator web application."
3. প্রযুক্তি স্ট্যাক (Tech Stack): "Next.js 14 App Router, Tailwind CSS, TypeScript, FastAPI backend, PostgreSQL with Prisma ORM, and Docker."
4. সুনির্দিষ্ট কাজ (Task & Constraints): "Create only the database schema and Dockerfile. Do not write frontend code yet. Follow strict security practices."
5. আউটপুট ফরম্যাট (Output Expectation): "Provide the complete file content, terminal commands, and explain what each configuration does."

🤖 গোল্ডেন প্রম্পট টেমপ্লেট (কপি করে ব্যবহার করুন)

```
Act as an expert Software Architect.
I am an Electrical Engineer building a [প্রজেক্টের নাম].
Tech Stack: [Next.js / Node.js / PostgreSQL / Docker]

Current Goal: [আপনি এখন যে নির্দিষ্ট ফিচারটি বানাতে চান]
Please design this step-by-step:
1. File structure and folder hierarchy.
2. Required dependencies and commands to install.
3. Complete code with comments explaining the logic.
4. How to test and verify locally in terminal.
Do not skip lines or use placeholders. Let's begin with Step 1.
```

১১.৩ ডিব্যাগিংয়ের মহৌষধ: স্ট্যাক ট্রেস (Stack Trace) পড়া শিখুন

কোড রান করার পর টার্মিনালে যখন ৫০ লাইনের লাল রঙের বিদঘুটে লেখা ভেসে ওঠে, সাধারণ মানুষ ভয় পেয়ে যান। কিন্তু একজন ইঞ্জিনিয়ার হিসেবে আপনাকে বুঝতে হবে স্ট্যাক ট্রেস কোনো অভিশাপ নয়, এটি অপরাধের ক্লু!

```
TypeError: Cannot read properties of undefined (reading 'email')
    at getUserProfile (/app/src/controllers/userController.js:45:22)
    at Layer.handle [as handle_request] (/app/node_modules/express/...
    at next (/app/node_modules/express/lib/router/route.js:144:13)
```

কীভাবে পড়বেন?

- প্রথম লাইন (Error Type & Reason): `TypeError: Cannot read properties of undefined (reading 'email')` — এর অর্থ হলো অবজেক্টটি খালি বা undefined, কিন্তু কোড তার ভেতরের `email` পড়তে চেয়েছে!
- দ্বিতীয় লাইন (Where it happened): `at getUserProfile (userController.js:45:22)` — ঠিক ৪৫ নম্বর লাইনের ২২ নম্বর কলামে দুর্ঘটনা ঘটেছে! মাঝের `node_modules` এর লাইনগুলো লাইব্রেরির ভেতরের কল, সেগুলো পড়ার কোনো দরকার নেই।

⚠️ এআই-কে এরর দেওয়ার সঠিক নিয়ম

কখনও বলবেন না "আমার কোড কাজ করছে না, ঠিক করে দাও"।
সবসময় টার্মিনালের সম্পূর্ণ লাল এরর লগ এবং যে ফাইলে এরর হয়েছে (যেমন `userController.js` এর পুরো কোড) একসাথে পেস্ট করবেন। এআই ২ সেকেন্ডে লাইন নম্বর দেখে আসল কারণ ধরিয়ে দেবে।

১১.৪ সিকিউরিটি ও এআই কোডিং সতর্কতা

1. কখনও এআই চ্যাটে আসল এপিআই কি দেবেন না: প্রম্পটে সবসময় `OPENAI_API_KEY="your-key-here"` বা ডামি ভ্যালু রাখুন। আসল চাবি কেবল আপনার লোকাল মেশিনের `.env` ফাইলে সংরক্ষণ করুন।
2. অন্ধের মতো কোড রান করবেন না: এআই যদি কোনো কমান্ড দেয় যাতে `rm -rf /` বা সিস্টেম ডিরেক্টরি ডিলিটের আভাস থাকে, কমান্ডটি বুঝে তবেই রান করবেন।

3. লুপে আটকে যাওয়া (Stuck in a Loop): অনেক সময় এআই একটি ভুল সমাধান দেয়, আপনি এরর দিলে সে আবার ঘুরিয়ে ফিরিয়ে একই ভুল কোড দিতে থাকে। এই অবস্থায় নতুন চ্যাট থ্রেড খুলুন এবং পুরো প্রবলেম নতুন করে সাজিয়ে দিন।

১১.৫ আইডিয়া থেকে প্রোডাকশন: ৭-ধাপের বাস্তব রোডম্যাপ

এআই দিয়ে যেকোনো নতুন ওয়েব বা মোবাইল সফটওয়্যার তৈরির আদর্শ পর্যায়ক্রম:

1. ধাপ ১ (আর্কিটেকচার ডিজাইন): এআই-এর সাথে ব্রেনস্টর্ম করে টেক স্ট্যাক ও ডাটাবেজ মডেল (Schema) ঠিক করুন।
2. ধাপ ২ (ডাটাবেজ ও ব্যাকএন্ড এপিআই): লোকাল ডাটাবেজ চালু করুন এবং মূল CRUD এপিআইগুলো তৈরি করে পোস্টম্যান (Postman) দিয়ে টেস্ট করুন।
3. ধাপ ৩ (ফ্রন্টএন্ড ইউআই): রিঅ্যাক্ট বা ফ্লাটার দিয়ে সুন্দর পেজ ডিজাইন করুন (প্রয়োজনে Tailwind CSS বা v0.dev ব্যবহার করুন)।
4. ধাপ ৪ (ইন্টিগ্রেশন): ফ্রন্টএন্ড থেকে ব্যাকএন্ড এপিআই কল করুন এবং ডেটা ডিসপ্লে করুন (CORS ও Auth হ্যান্ডল করুন)।
5. ধাপ ৫ (ডকারাইজেশন): `Dockerfile` ও `docker-compose.yml` তৈরি করে এক ক্লিকে পুরো অ্যাপ লোকালি রান করুন।
6. ধাপ ৬ (সিআই/সিডি অটোমেশন): GitHub Actions দিয়ে টেস্ট ও লিন্টিং অটোমেট করুন।
7. ধাপ ৭ (লাইভ ডেপ্লয়মেন্ট): Vercel (Frontend) ও Render/VPS (Backend/DB) এ লাইভ করে ডোমেন এবং SSL যুক্ত করুন!

> 📌 **অধ্যায় ১১ এর সারসংক্ষেপ**
>
> - এআই-কে একক বিশাল কাজের বদলে আর্কিটেক্ট ফ্রেমওয়ার্কে ছোট ছোট সুনির্দিষ্ট টাস্ক দিন।
> - স্ট্যাক ট্রেসের ১ম লাইন (কারণ) এবং ২য় লাইন (ফাইলের নাম ও লাইন নম্বর) হলো ডিব্যাগিংয়ের সোনার কাঠি।
> - কখনই এআই প্রম্পটে আসল পাসওয়ার্ড বা প্রাইভেট কি পেস্ট করবেন না; ব্যবহার করুন `.env`।
> - ৭-ধাপের রোডম্যাপ (আর্কিটেকচার → ডাটাবেজ → ফ্রন্টএন্ড → ইন্টিগ্রেশন → ডকার → সিআই/সিডি → ক্লাউড) অনুসরণ করলে যেকোনো জটিল সফটওয়্যার সফলভাবে নির্মাণ সম্ভব।

CHAPTER 12

সিএস পরিভাষার পূর্ণাঙ্গ অভিধান (100+ Essential Terms Glossary)

দৈনন্দিন সফটওয়্যার ডেভেলপমেন্ট ও এআই চ্যাটের বহুল ব্যবহৃত টেকনিক্যাল শব্দের সহজ বাংলা অর্থ

সফটওয়্যার জগতে প্রতিদিন শত শত টেকনিক্যাল টার্মের মুখোমুখি হতে হয়। নিচে ১০০টি সবচেয়ে গুরুত্বপূর্ণ পরিভাষার বর্ণানুক্রমিক (A-Z) সহজ অর্থ ও ব্যাখ্যা দেওয়া হলো:

শব্দ (Term)	সহজ বাংলা ব্যাখ্যা ও প্রায়োগিক অর্থ
API (Application Programming Interface)	দুটি ভিন্ন সফটওয়্যারের মধ্যে তথ্য ও কমান্ড আদান-প্রদানের নিয়মমাফিক ডিজিটাল সংযোগ সেতু।
Authentication (AuthN)	ইউজার আসলেই কে তা প্রমাণ করা (যেমন: ইউজারনেম ও পাসওয়ার্ড বা বায়োমেট্রিক দিয়ে লগইন)।
Authorization (AuthZ)	লগইন করা ইউজারের কোন কোন ফাইল বা ফিচার দেখার অধিকার (Permission) আছে তা নির্ধারণ করা (যেমন: ইউজার বনাম অ্যাডমিন)।
Boilerplate Code	প্রজেক্ট শুরু করার জন্য প্রয়োজনীয় প্রাথমিক কাঠামো বা কোড টেমপ্লেট যা প্রতিটি প্রজেক্টেই বারবার কপি করতে হয়।
Bug & Debugging	সফটওয়্যারের ত্রুটি বা অপ্রত্যাশিত আচরণকে বলে 'বাগ' (Bug), আর সেই ত্রুটি খুঁজে সমাধান করার প্রক্রিয়া হলো 'ডিবাগিং'।
Cache (ক্যাশ)	ঘন ঘন ব্যবহৃত ডেটাকে অতি দ্রুতগতির মেমোরিতে (RAM / Redis) সাময়িকভাবে ধরে রাখা যাতে বারবার মূল ডাটাবেজে যেতে না হয়।
CI/CD (Continuous Integration / Continuous Deployment)	কোড পুশ করলে স্বয়ংক্রিয়ভাবে টেস্ট করা এবং সার্ভারে কোনো মানুষের স্পর্শ ছাড়াই লাইভ ডেপ্লয় করার আধুনিক পাইপলাইন।
Client & Server	ক্লায়েন্ট হলো ইউজার ডিভাইস (ব্রাউজার বা মোবাইল অ্যাপ) যা সার্ভিস চায়; সার্ভার হলো কেন্দ্রীয় শক্তিশালী মেশিন যা সার্ভিস প্রদান করে।
CORS (Cross-Origin Resource Sharing)	ব্রাউজারের একটি নিরাপত্তা নিয়ম যা এক ডোমেনের ফ্রন্টএন্ডকে অন্য ডোমেনের ব্যাকএন্ডের ডেটা অ্যাক্সেস করতে অনুমতি দেয় বা বাধা দেয়।
CRUD (Create, Read, Update, Delete)	যেকোনো ডাটাবেজ বা সফটওয়্যারের মৌলিক ৪টি কাজ: নতুন ডেটা তৈরি, পড়া, পরিবর্তন এবং মুছে ফেলা।
Daemon (ডেমন)	অপারেটিং সিস্টেমের ব্যাকগ্রাউন্ডে কোনো ব্যবহারকারী ইন্টারফেস ছাড়াই নিরবচ্ছিন্নভাবে চলতে থাকা বিশেষ প্রসেস বা সার্ভিস।
Database Migration	ডাটাবেজের স্কিমা বা টেবিলের গঠন পরিবর্তনের ইতিহাস কোড আকারে সংরক্ষণ ও স্বয়ংক্রিয়ভাবে কার্যকর করার সিস্টেম।
DevOps (Development + Operations)	সফটওয়্যার তৈরি (Dev) এবং সার্ভারে তা রক্ষণাবেক্ষণ ও অটোমেশন (Ops) করার যৌথ আধুনিক ইঞ্জিনিয়ারিং কালচার।
DNS (Domain Name System)	ইন্টারনেটের ফোনবুক, যা মানুষের পঠনযোগ্য ডোমেন নামকে (যেমন google.com) কম্পিউটারের আইপি অ্যাড্রেসে রূপান্তর করে।
Docker Container	একটি অ্যাপ্লিকেশনের কোড, রানটাইম ও সব লাইব্রেরিসহ সিল করা একটি হালকা ভার্চুয়ালাইজড এক্সিকিউশন প্যাকেজ।
Dockerfile	ডকার ইমেজ তৈরি করার ধাপে ধাপে নির্দেশনা সম্বলিত একটি সাধারণ টেক্সট ফাইল বা ব্লুপ্রিন্ট।

DOM (Document Object Model)	ব্রাউজারের মেমোরিতে থাকা একটি এইচটিএমএল পেজের গাছরূপ (Tree structure), যা জাভাস্ক্রিপ্ট দিয়ে পরিবর্তন করা যায়।
Endpoint (এন্ডপয়েন্ট)	একটি নির্দিষ্ট সার্ভার ইউআরএল (যেমন `/api/v1/products`) যার মাধ্যমে কোনো নির্দিষ্ট তথ্য বা সার্ভিস পাওয়া যায়।
Environment Variables (.env)	অপারেটিং সিস্টেম লেভেলে সংরক্ষিত ভ্যারিয়েবল যা ডাটাবেজ পাসওয়ার্ড বা এপিআই সিক্রেট কি কোডের বাইরে সুরক্ষিত রাখে।
Frontend & Backend	ফ্রন্টএন্ড হলো ইউজারের চোখের সামনের দৃশ্যমান ইন্টারফেস; ব্যাকএন্ড হলো সার্ভারে চলা অদৃশ্য লজিক, সিকিউরিটি ও ডাটা ইঞ্জিন।
Fullstack Developer	যে ব্যক্তি একই সাথে ক্লায়েন্ট সাইড (ফ্রন্টএন্ড), সার্ভার সাইড (ব্যাকএন্ড), ডাটাবেজ ও ডেপ্লয়মেন্টের কাজ সম্পন্ন করতে পারেন।
Garbage Collection (GC)	মেমোরিতে যে অবজেক্টগুলো আর কখনো ব্যবহার হবে না, অপারেটিং সিস্টেম বা রানটাইম স্বয়ংক্রিয়ভাবে তা মুছে র‍্যাম খালি করার প্রক্রিয়া।
Git & GitHub	গিট হলো অফলাইন ভার্সন কন্ট্রোল সফটওয়্যার; গিটহাব হলো ইন্টারনেটে গিট রিপোজিটরি হোস্ট করার জনপ্রিয় ক্লাউড প্ল্যাটফর্ম।
GraphQL	ফেসবুকের তৈরি এপিআই কোয়েরি ল্যাঙ্গুয়েজ, যাতে ক্লায়েন্ট নিজের ইচ্ছামতো শুধু প্রয়োজনীয় ফিল্ডের ডেটা চাইতে পারে।
Hot Reloading	কোড এডিট করে সেভ করার সাথে সাথে পুরো পেজ রিলোড না করেই ব্রাউজারে বা অ্যাপে তাৎক্ষণিক ফলাফল দেখতে পাওয়া।
HTTP / HTTPS	ইন্টারনেটে ওয়েবপেজ ও ডেটা আদান-প্রদানের মৌলিক প্রোটোকল; 'S' মানে সিকিউরড বা এনক্রিপ্টেড (SSL/TLS)।
Idempotent (আইডেমপোটেন্ট)	এমন একটি অপারেশন যা সার্ভারে একবার রান করলেও যে ফলাফল হয়, শতবার রান করলেও একই ফলাফল বহাল থাকে (যেমন: HTTP GET বা PUT)।
JSON (JavaScript Object Notation)	সহজ টেক্সট-ভিত্তিক কি-ভ্যালু ডেটা ফরম্যাট যা আজকের বিশ্বের প্রায় সব এপিআই ও প্রোগ্রামিং ভাষা সমর্থন করে।
JWT (JSON Web Token)	ক্রিপ্টোগ্রাফিক সইযুক্ত একটি কম্প্যাক্ট টোকেন যা দিয়ে সার্ভারলেস ও স্ট্যাটলেস উপায়ে ব্যবহারকারীর পরিচয় যাচাই করা হয়।
Kubernetes (K8s)	হাজার হাজার ডকার কন্টেইনারকে স্বয়ংক্রিয়ভাবে ক্লাউডে স্কেলিং ও সমন্বয় করার ওপেন সোর্স অর্কেস্ট্রেশন ইঞ্জিন।
Linting / Linter	কোড রান না করেই স্ট্যাটিক অ্যানালাইসিস করে কোডের স্পেলিং ভুল, বাজে স্টাইল বা সিনট্যাক্স ত্রুটি ধরিয়ে দেওয়া (যেমন: ESLint)।
Load Balancer	হাজার হাজার ইউজারের ট্রাফিক একাধিক সমান্তরাল সার্ভারের মাঝে সুষমভাবে ভাগ করে দেওয়ার ডিভাইস বা সফটওয়্যার।
Middleware	

	একটি ক্লায়েন্ট রিকোয়েস্ট মূল ব্যাকএন্ড কন্ট্রোলারে পৌঁছানোর ঠিক মাঝপথে ডেটা ফিল্টার বা অথেনটিকেশন চেক করার কোড স্তর।
Microservices	একটি বিশাল সফটওয়্যারকে ক্ষুদ্র ক্ষুদ্র স্বাধীন মডিউলে ভাগ করে আলাদা আলাদা ডেপ্লয় করার আধুনিক আর্কিটেকচার।
Monolith	একটিমাত্র কোডবেস এবং একক সার্ভারে চলা সম্পূর্ণ সফটওয়্যার সিস্টেম।
Nginx	বিশ্বখ্যাত উচ্চগতির ওপেন সোর্স ওয়েব সার্ভার, রিভার্স প্রক্সি এবং লোড ব্যালেন্সার।
NoSQL	ঐতিহ্যবাহী টেবিল কাঠামোর বাইরে গিয়ে JSON ডকুমেন্ট, কি-ভ্যালু বা গ্রাফ মডেলে ডেটা রাখার ডাটাবেজ (যেমন MongoDB)।
Node.js	গুগলের V8 জাভাস্ক্রিপ্ট ইঞ্জিনের ওপর তৈরি রানটাইম, যা ব্রাউজারের বাইরে সার্ভারে জাভাস্ক্রিপ্ট রান করার সুযোগ দেয়।
Object Storage	ফাইল, ছবি বা ভিডিওকে ফ্ল্যাট বাকেটে ক্লাউডে সংরক্ষণ করার সিস্টেম (যেমন AWS S3 বা Cloudflare R2)।
ORM (Object-Relational Mapping)	এসকিউএল কোড না লিখে নিজের প্রোগ্রামিং ল্যাঙ্গুয়েজের সাধারণ অবজেক্ট বা মেথড দিয়ে ডাটাবেজ পরিচালনার লাইব্রেরি (যেমন Prisma)।
Package Manager	ইন্টারনেট থেকে থার্ড-পার্টি লাইব্রেরি ও ডিপেন্ডেন্সি ইনস্টল ও পরিচালনা করার স্বয়ংক্রিয় টুল (যেমন npm, pip)।
Payload	একটি নেটওয়ার্ক রিকোয়েস্ট বা প্যাকেটের ভেতরে থাকা মূল প্রয়োজনীয় ডেটা (যেমন POST রিকোয়েস্টের ফর্ম ডেটা)।
Pull Request (PR)	একটি ব্রাঞ্চের নতুন কোড মূল প্রজেক্টে যুক্ত (Merge) করার জন্য টিম লিডারের কাছে আনুষ্ঠানিক অনুমোদনের আবেদন।
Rate Limiting	একটি নির্দিষ্ট আইপি বা ইউজার প্রতি মিনিটে সর্বোচ্চ কতবার এপিআই কল করতে পারবে তার সীমা বেঁধে দেওয়া (DDoS রোধে)।
Redis	অত্যন্ত দ্রুতগতির ইন-মেমোরি কি-ভ্যালু স্টোর যা প্রধানত হাই-পারফরম্যান্স ক্যাশিং ও সেশন ম্যানেজমেন্টে ব্যবহৃত হয়।
REST API	ওয়েব আর্কিটেকচারের একটি সুনির্দিষ্ট স্ট্যান্ডার্ড যাতে HTTP মেথডস (GET, POST, PUT, DELETE) দিয়ে ডেটা হ্যান্ডল করা হয়।
Reverse Proxy	বাইরের ট্রাফিক সরাসরি মূল অ্যাপ সার্ভারে না দিয়ে সামনে বসে সব রিকোয়েস্ট গ্রহণ ও ভেতরে রাউট করার নিরাপত্তা ঢাল।
Root / Sudo	লিনাক্স অপারেটিং সিস্টেমের সর্বোচ্চ ক্ষমতাধর অ্যাডমিনিস্ট্রেটর বা সুপার-ইউজার পারমিশন।
Routing	কোন ইউআরএল (URL) লিংকে ঢুকলে কোন পেজ বা কোন ব্যাকএন্ড ফাংশন কার্যকর হবে তা নির্ধারণ করার লজিক।
SemVer (Semantic Versioning)	সফটওয়্যার ভার্সন লেখার বৈশ্বিক নিয়ম: MAJOR.MINOR.PATCH (যেমন v2.1.4)।

Serverless	ক্লাউড কম্পিউটিং মডেল যেখানে ডেভেলপারকে কোনো সার্ভার কিনতে বা কনফিগার করতে হয় না, কোড প্রতি এক্সিকিউশন অনুযায়ী রান হয়।
Session & Cookies	ব্যবহারকারীর লগইন অবস্থা ব্রাউজারের কুকিতে ও সার্ভারের মেমোরিতে ধরে রাখার পদ্ধতি।
SQL (Structured Query Language)	রিলেশনাল ডাটাবেজ টেবিল থেকে তথ্য খোঁজা ও সংরক্ষণ করার আন্তর্জাতিক স্ট্যান্ডার্ড কোয়েরি ল্যাঙ্গুয়েজ।
SSH (Secure Shell)	ইন্টারনেটের মাধ্যমে দূরবর্তী লিনাক্স সার্ভারে এনক্রিপ্টেড টার্মিনাল লগইন ও নিয়ন্ত্রণের স্ট্যান্ডার্ড প্রোটোকল।
Stack Overflow	মেমোরির স্ট্যাক সীমা ছাড়িয়ে গেলে ঘটা ক্র্যাশ; একই সাথে ডেভেলপারদের বিখ্যাত প্রশ্নোত্তরের ওয়েবসাইট।
Stack Trace	কোডে কোনো এরর ঘটলে কোন ফাইলের কত নম্বর লাইন দিয়ে দুর্ঘটনাটি ঘটেছে তার ধারাবাহিক ট্র্যাকিং লিস্ট।
Synchronous vs Asynchronous	সিঙ্ক্রোনাস মানে এক কাজ শেষ না হওয়া পর্যন্ত পরের কাজ অপেক্ষা করবে; অ্যাসিনক্রোনাস মানে ব্যাকগ্রাউন্ডে কাজ চলতে থাকবে এবং মূল সিস্টেম আটকে থাকবে না।
Tailwind CSS	ইউটিলিটি-ফার্স্ট সিএসএস ফ্রেমওয়ার্ক যা দিয়ে সরাসরি এইচটিএমএল ট্যাগের ভেতরে ক্লাস লিখে অসাধারণ দ্রুত ডিজাইন করা যায়।
TypeScript	জাভাস্ক্রিপ্টের ওপর তৈরি মাইক্রোসফটের স্ট্রংলি-টাইপড ল্যাঙ্গুয়েজ যা কোড রান করার আগেই টাইপ ভুলের বাগ ধরে ফেলে।
UUID (Universally Unique Identifier)	১২৮-বিটের এমন একটি অনন্য আইডি (যেমন: `67913c52-1d54-4f6c-a082-ed4c6ebaaa38`) যা পৃথিবীতে কখনো ডুপ্লিকেট হবে না।
Vercel	আধুনিক ওয়েব অ্যাপ ও ফ্রন্টএন্ড (বিশেষ করে Next.js) গিটহাব থেকে সরাসরি এক ক্লিকে লাইভ হোস্ট করার বিশ্বসেরা প্ল্যাটফর্ম।
Virtual DOM	রিঅ্যাক্টের নিজস্ব মেমোরি রিপ্রেজেন্টেশন যা দ্রুত পরিবর্তন হিসেব করে এবং আসল ব্রাউজার পেজে শুধু ক্ষুদ্রাতিক্ষুদ্র পরিবর্তন ঘটায়।
VPS (Virtual Private Server)	ক্লাউডে সম্পূর্ণ রুট অ্যাক্সেসসহ ভাড়া নেওয়া একটি স্বাধীন লিনাক্স ভার্চুয়াল কম্পিউটার (যেমন DigitalOcean Droplet)।
WebSocket	সার্বক্ষণিক উন্মুক্ত বাই-ডিরেকশনাল সকেট যোগাযোগ যা দিয়ে চোখের পলকে চ্যাট ও লাইভ নোটিফিকেশন পাঠানো যায়।
XSS (Cross-Site Scripting)	একটি মারাত্মক ওয়েব সিকিউরিটি বাগ যেখানে হ্যাকার ক্ষতিকর জাভাস্ক্রিপ্ট ইনজেক্ট করে অন্যের কুকি বা টোকেন চুরি করে।
YAML (Yet Another Markup Language)	মানুষের পড়ার উপযোগী অতি পরিষ্কার ডেটা কনফিগারেশন ফরম্যাট যা ডকার কম্পোজ ও গিটহাব অ্যাকশনসে বহুল ব্যবহৃত।

🎉 অভিনন্দন! আপনি এখন সফটওয়্যার আর্কিটেক্ট!

এই ১২টি অধ্যায়ে আপনি সফটওয়্যারের প্রতিটি পরত—কম্পিউটার ফান্ডামেন্টালস থেকে শুরু করে ব্রাউজার, ব্যাকএন্ড, নেটওয়ার্কিং, ডাটাবেজ, গিট, ডকার, গিটহাব অ্যাকশনস, ক্লাউড ডেপ্লয়মেন্ট এবং এআই প্রম্পটিং নিখুঁতভাবে আয়ত্ত করেছেন।

এখন আপনি যখন Cursor, Claude বা ChatGPT খুলে বলবেন: "Build me a fullstack app with Next.js, FastAPI, PostgreSQL, Docker, and GitHub Actions CI/CD pipeline"—আপনি এর প্রতিটি শব্দের অর্থ, তাদের পারস্পরিক সংযোগ এবং প্রতিটি সম্ভাব্য এররের কারণ জানবেন।

আপনার উদ্ভাবনী নতুন প্রজেক্টের যাত্রায় শুভকামনা!